

Machine Learning for Systems and Control

5SC28

Lecture 3

dr. ir. Maarten Schoukens & prof. dr. ir. Roland Tóth

Control Systems Group

Department of Electrical Engineering

Eindhoven University of Technology

Academic Year: 2025-2026 (version 28/04/2026)

Learning Objectives

Deep Learning & Deep Neural Networks

Training a Deep Neural Network

Artificial Neural Networks for Dynamical Systems

Artificial Neural Networks

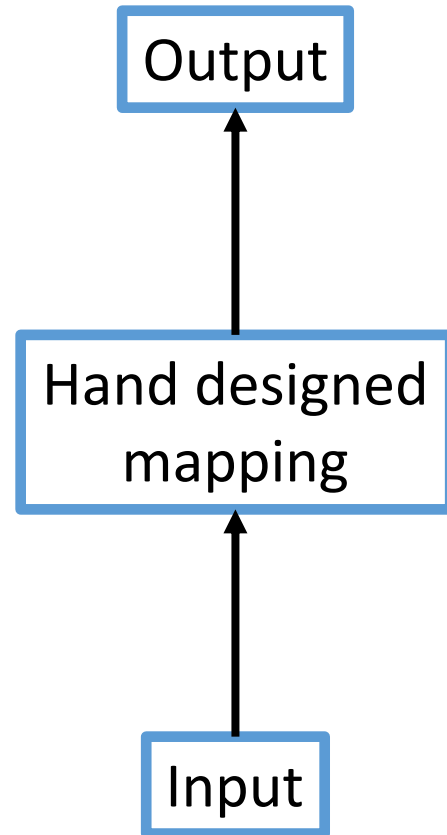
Deep Learning & Deep Neural Networks

Training a Deep Neural Network

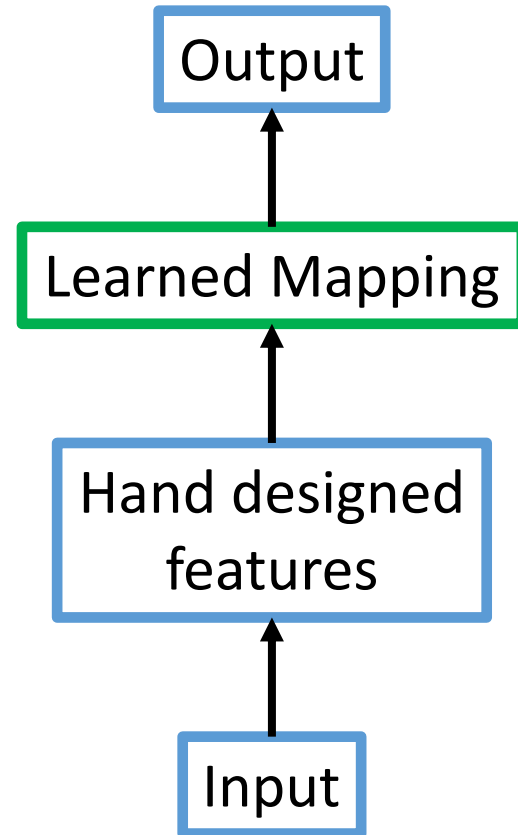
Artificial Neural Networks for Dynamical Systems

Deep Learning

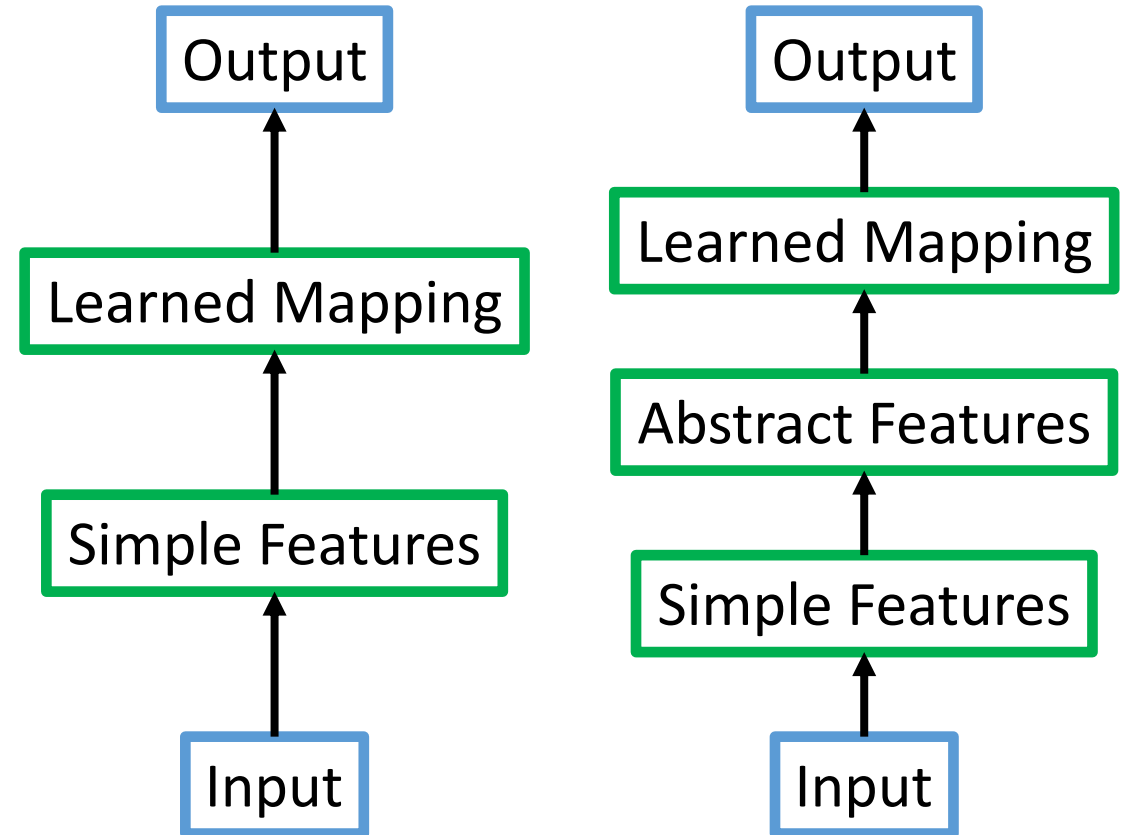
Rule-Based Systems



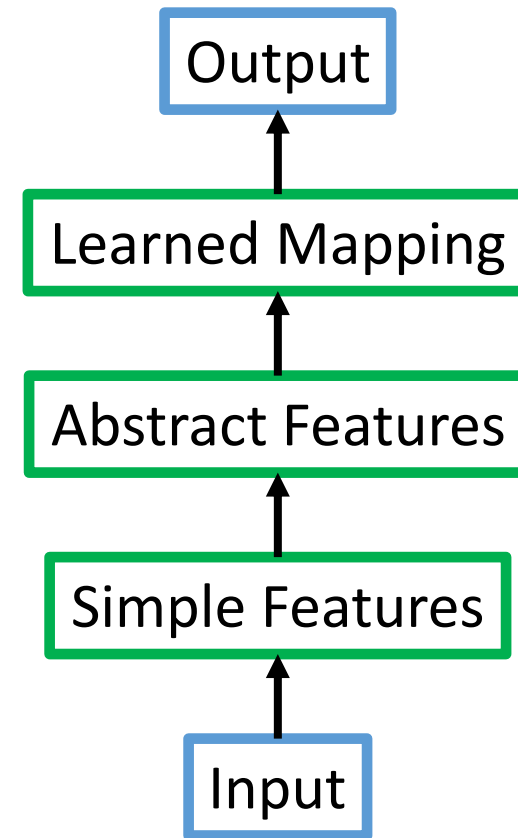
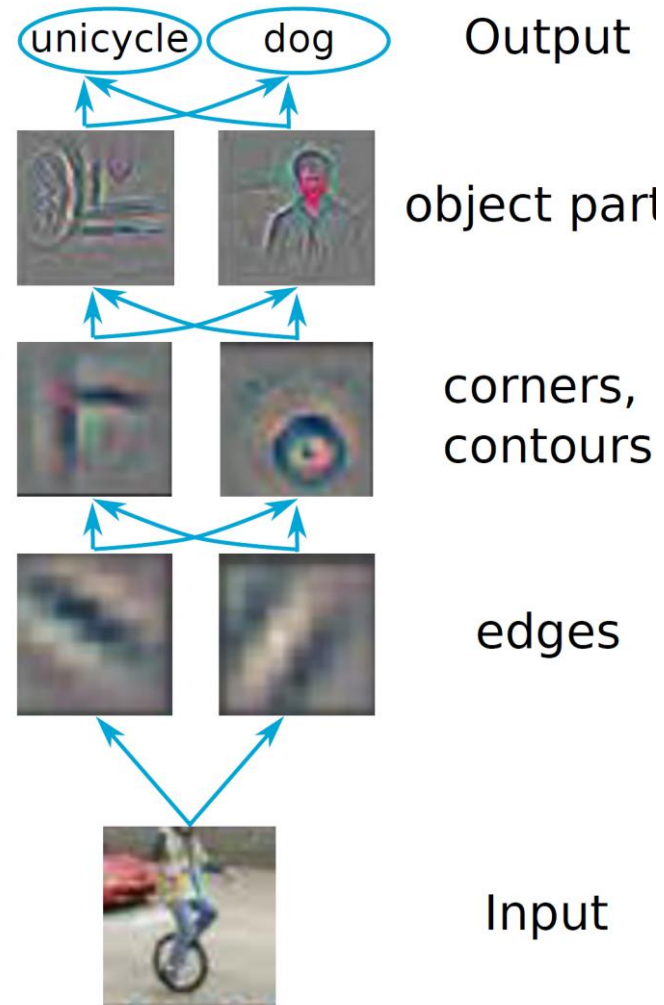
Classical Learning



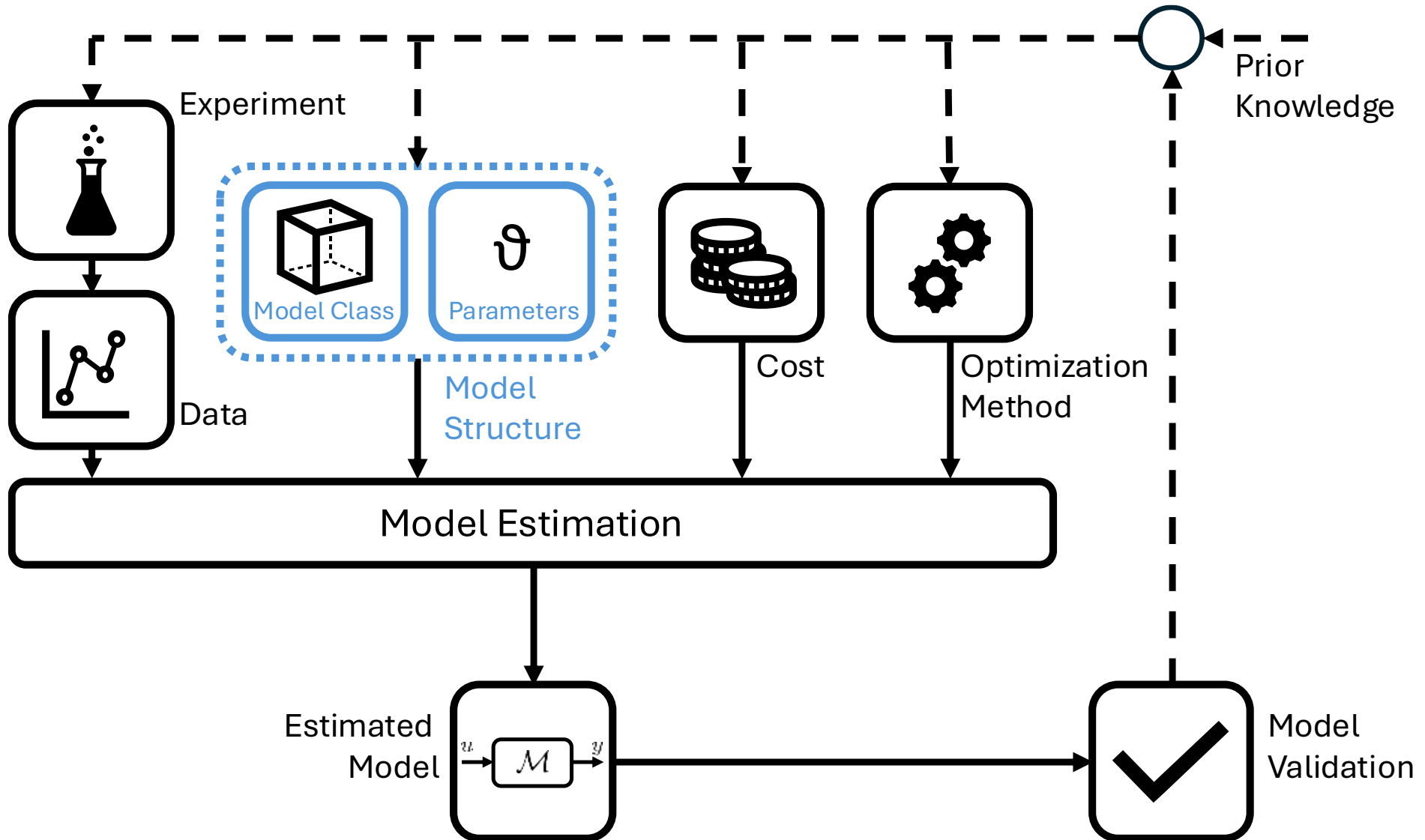
Deep Learning



Deep Learning

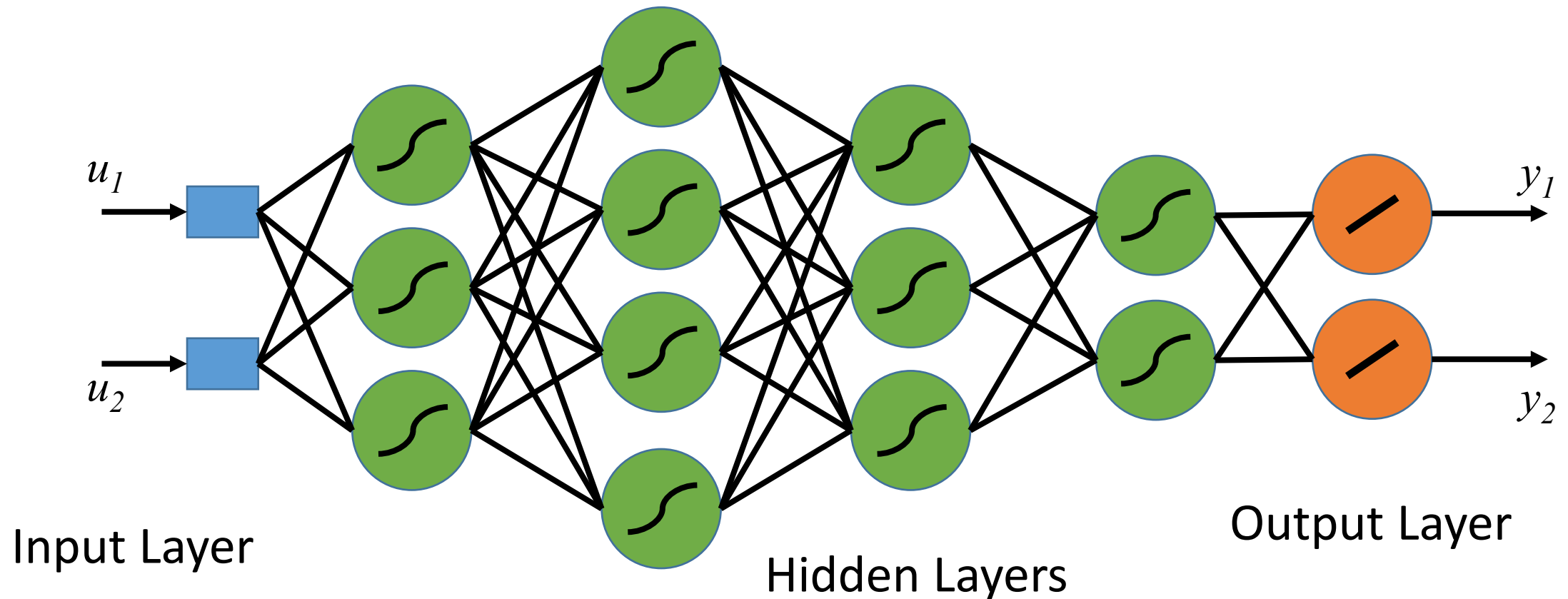


Data-Driven Modelling Process



Deep Neural Network

“A network with multiple hidden layers”¹



¹ Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

Deep Neural Networks

Deep Feedforward

Recurrent

Long-Short Term Memory

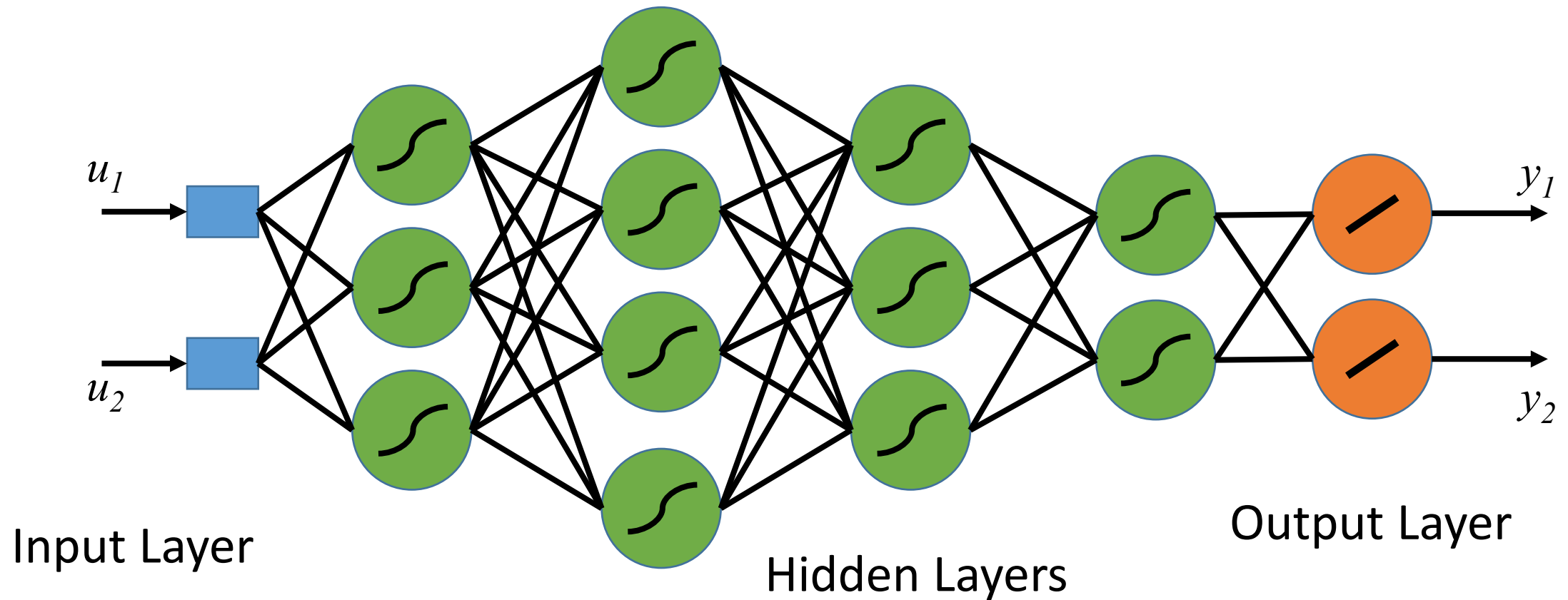
Autoencoder

Residual

Convolutional

...

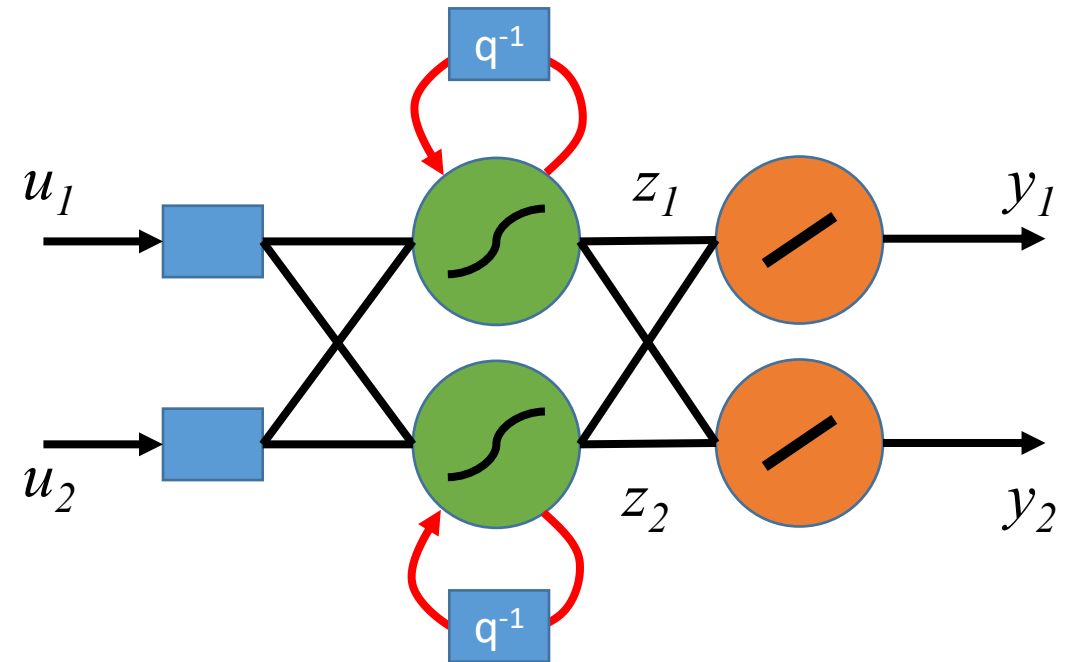
Deep Feedforward Neural Network



Recurrent Neural Networks

Multiple recurrence schemes possible

$$z_j(k) = g(w_{1,j}^\top u(k) + v_{1,j}z_j(k-1) + b_{1,j})$$
$$y_j(k) = w_{2,j}^\top z(k) + b_{2,j}$$

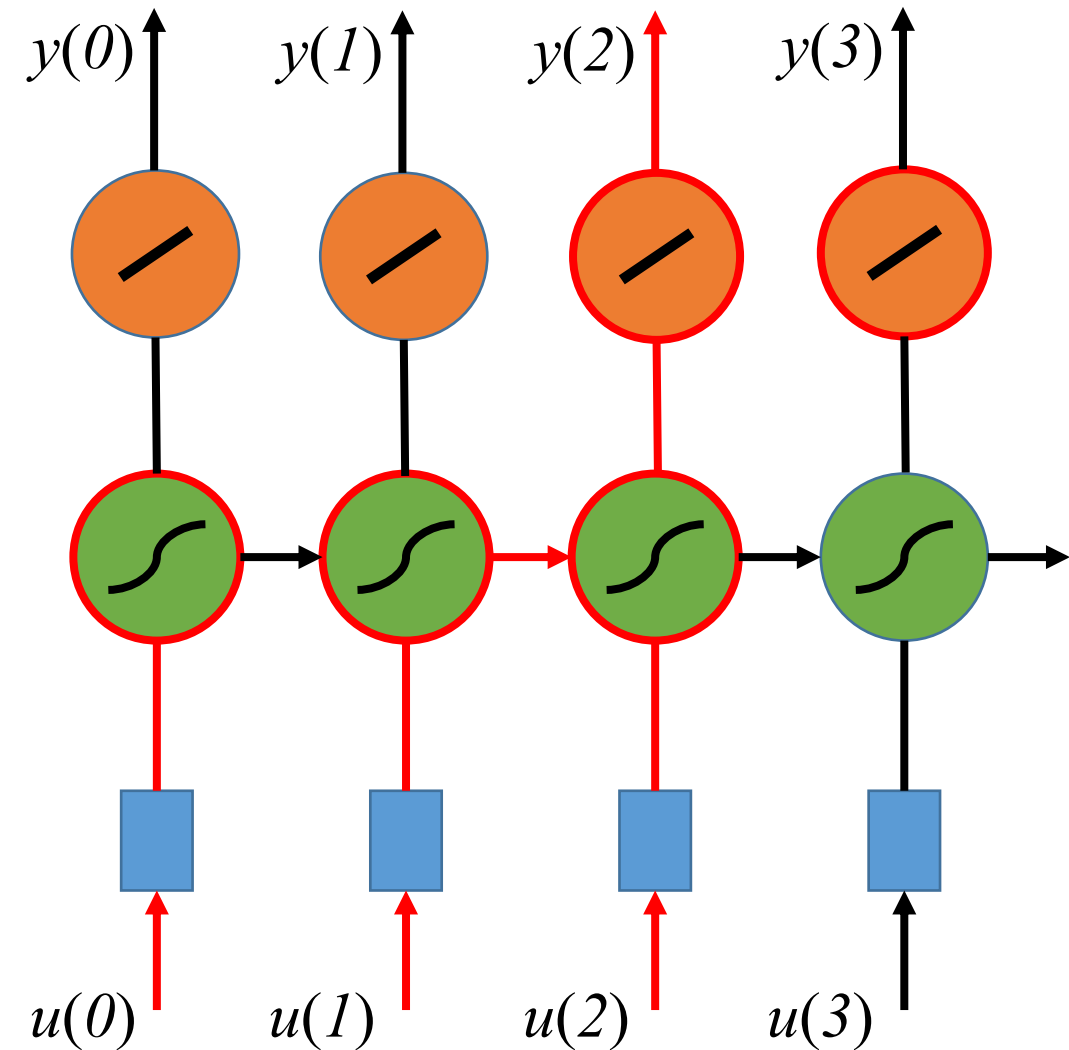
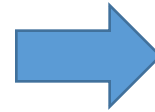
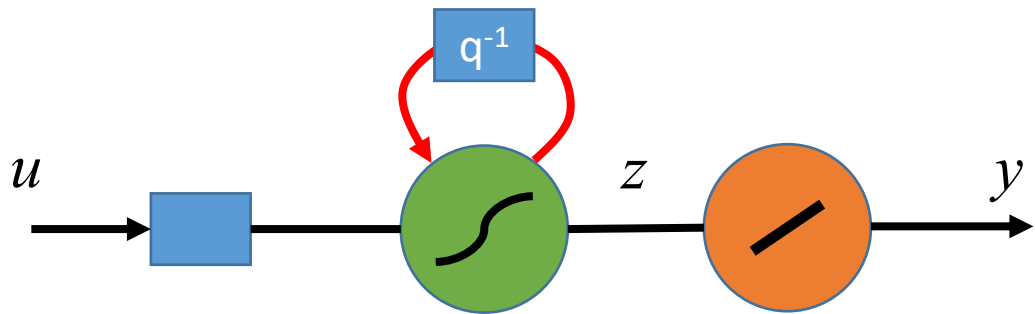


Introduces states in the network
The mapping from u to y is now dynamic

Recurrent Neural Networks - Training

Network Unfolding

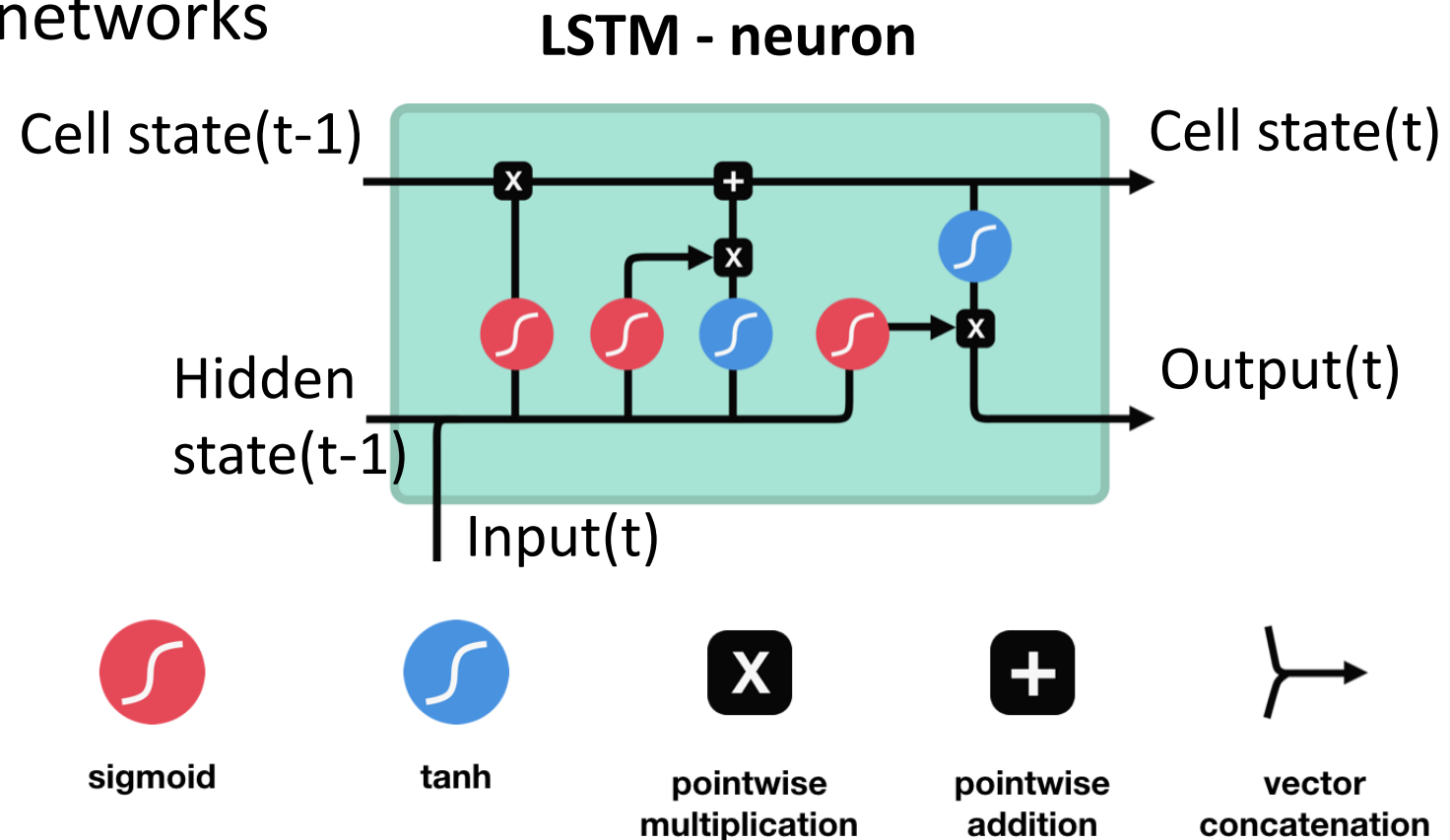
Backpropagation Through Time



Long-Short Term Memory Networks

Extension of recurrent neural networks

For experiences with
very long time lags



Long-Short Term Memory Networks

Extension of recurrent neural networks

For experiences with
very long time lags

Input gate:

Decide when to update **memory**

Output gate:

Decide when to output **memory**

Forget Gate:

Decide when to erase **memory**



sigmoid



tanh



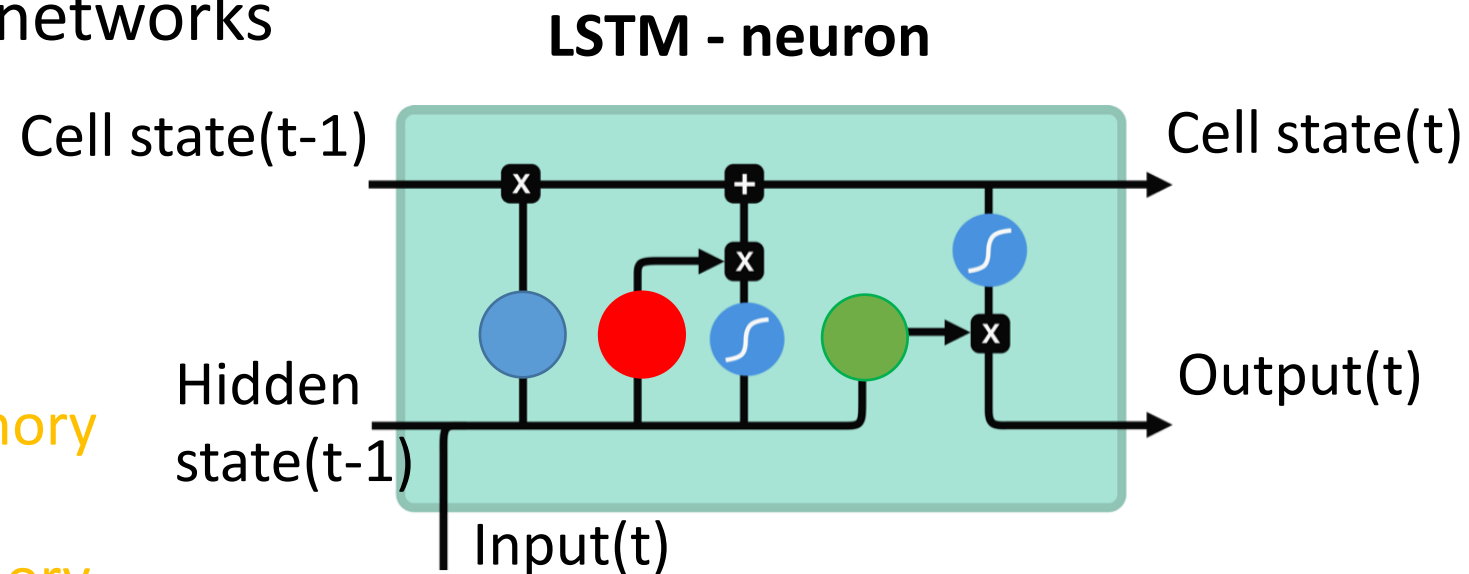
pointwise
multiplication



pointwise
addition



vector
concatenation



Long-Short Term Memory Networks

Extension of recurrent neural networks

For experiences with
very long time lags

Input gate:

Decide when to update **memory**

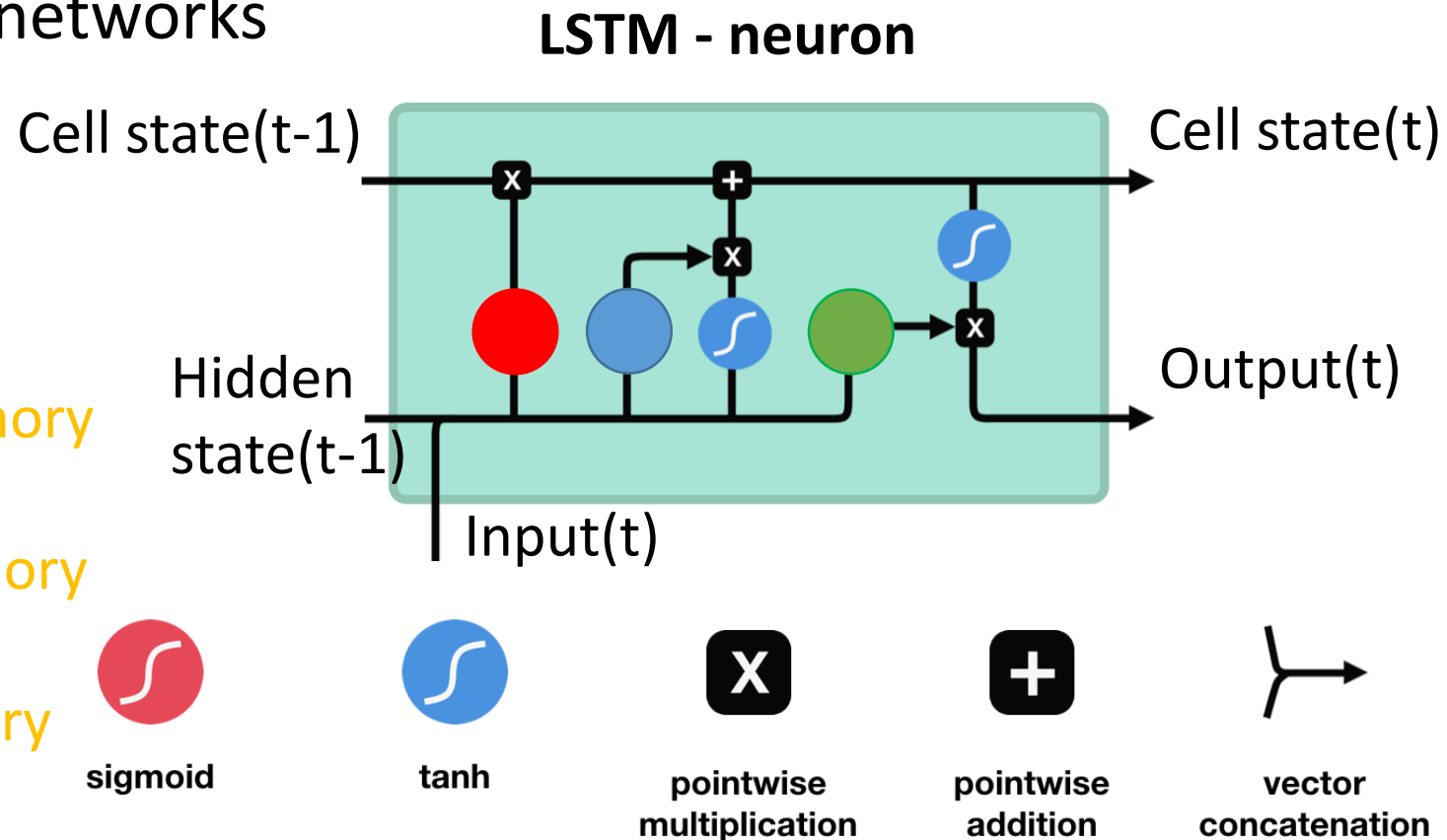
Output gate:

Decide when to output **memory**

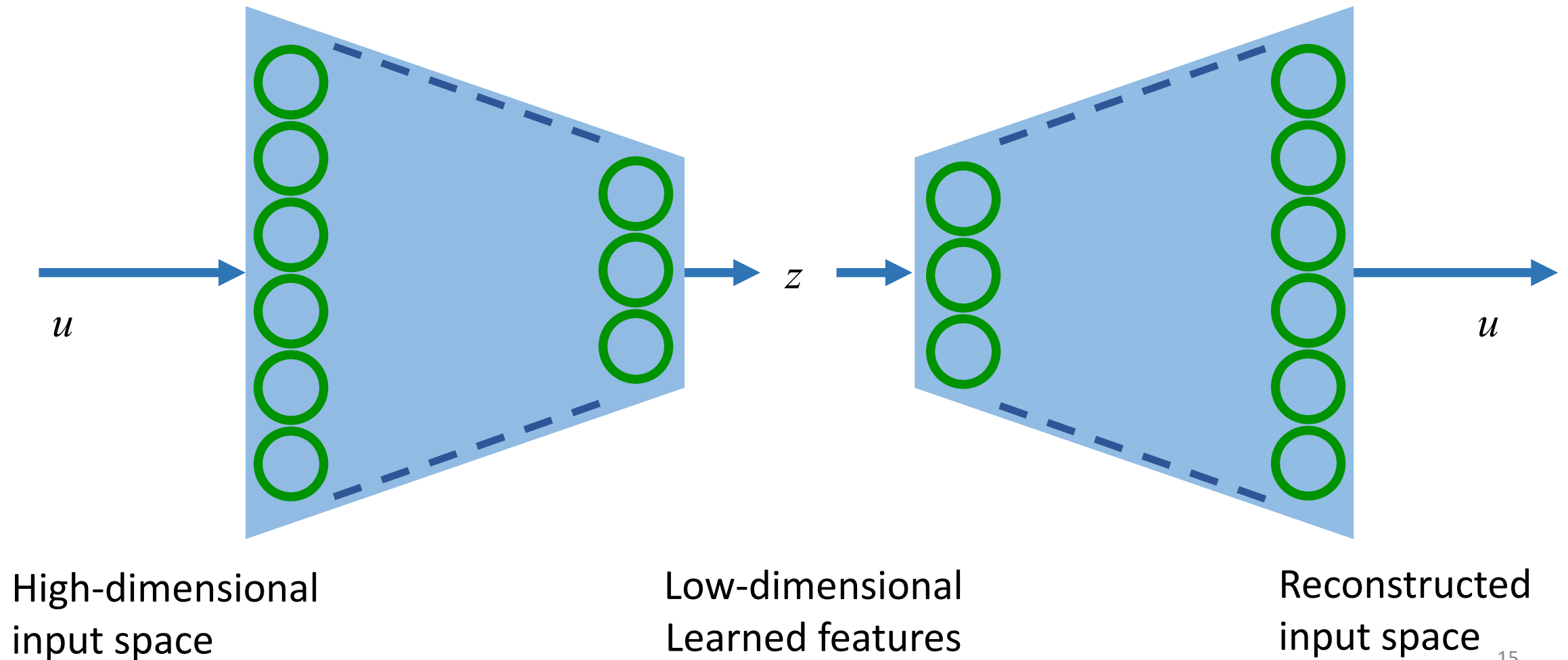
Forget Gate:

Decide when to erase **memory**

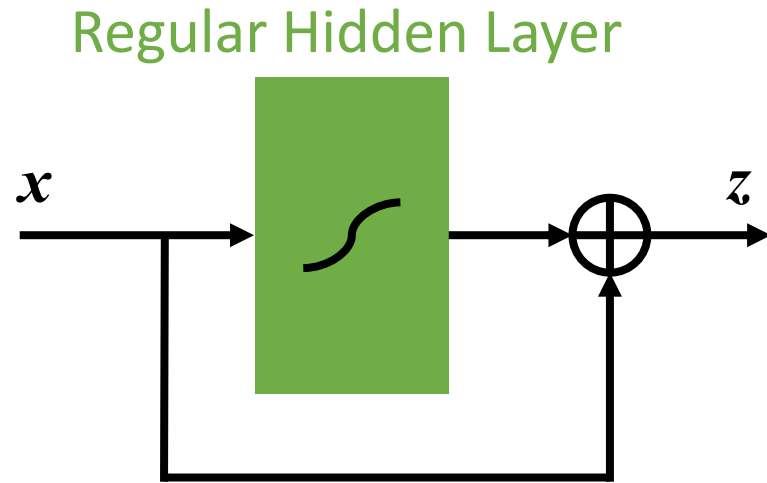
Alternative: GRU



Auto-Encoder Neural Networks



Residual Networks



Residual Network Layer

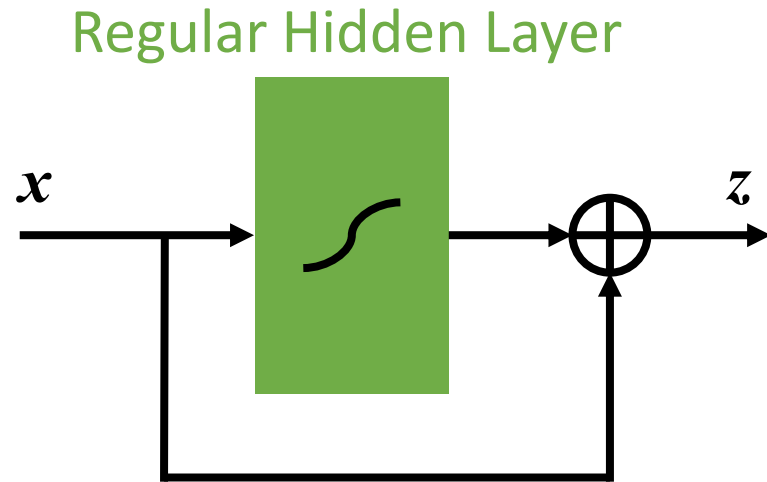
$$z = x + g(W_1x + b_1)$$

Improved training characteristics

Direct feedthrough can skip multiple layers

Dimension z = dimension x

Residual Networks: Changing Dimensions



Residual Network Layer

$$z = W_s x + g(W_1 x + b_1)$$

Improved training characteristics

Direct feedthrough can skip multiple layers

Dimension $z \neq$ dimension x

- Zero-padding for increasing dimensions
- Linear projection for changing output dimensions (e.g. by 1x1 convolution)

Convolutional Networks

Convolve the input with a filter

Learn filter weights + bias

Spatial / Temporal relation of entries preserved
(as opposed to vectorizing the tensor / matrix)

Layer output decreases in dimension → perform padding to preserve the same dimension

Convolutional Operation

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature

source: <https://towardsdatascience.com/>

Convolutional Networks – Pooling

Dimension reduction

Noise suppression

Extract dominant features

Max or average pooling

Max Pooling

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

source: <https://towardsdatascience.com/>

Deep Neural Networks

Deep Feedforward

basic structure

Recurrent

time series, natural language processing

Long-Short Term Memory

time series, natural language processing (long dependencies)

Autoencoder

dimension reduction, feature learning

Residual

to go deep

Convolutional

image / video processing, spatial-temporal

Structured State-Space Models

time series, natural language processing (long dependencies)

Transformers

time series, natural language processing (long dependencies)

...

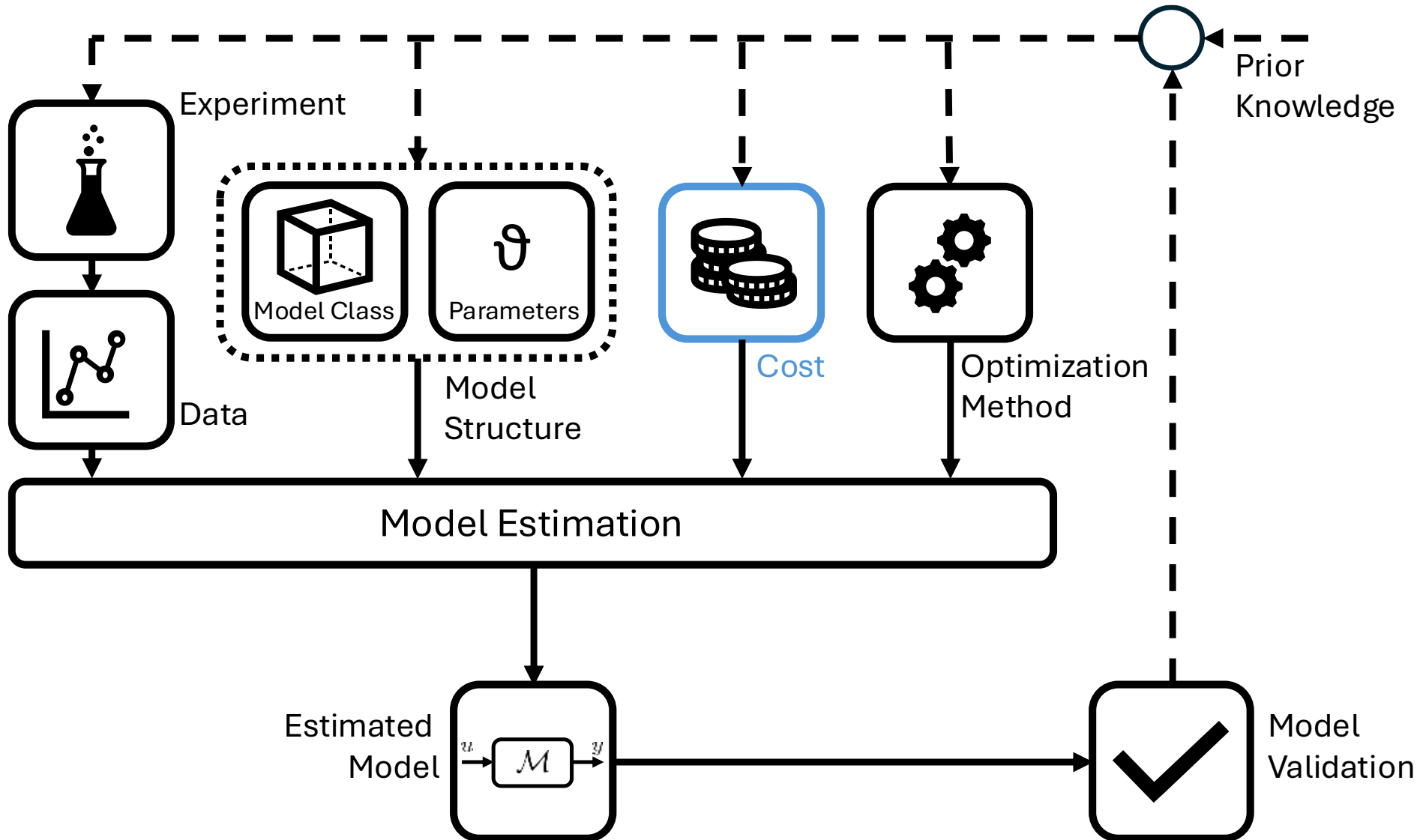
Artificial Neural Networks

Deep Learning & Deep Neural Networks

Training a Deep Neural Network

Artificial Neural Networks for Dynamical Systems

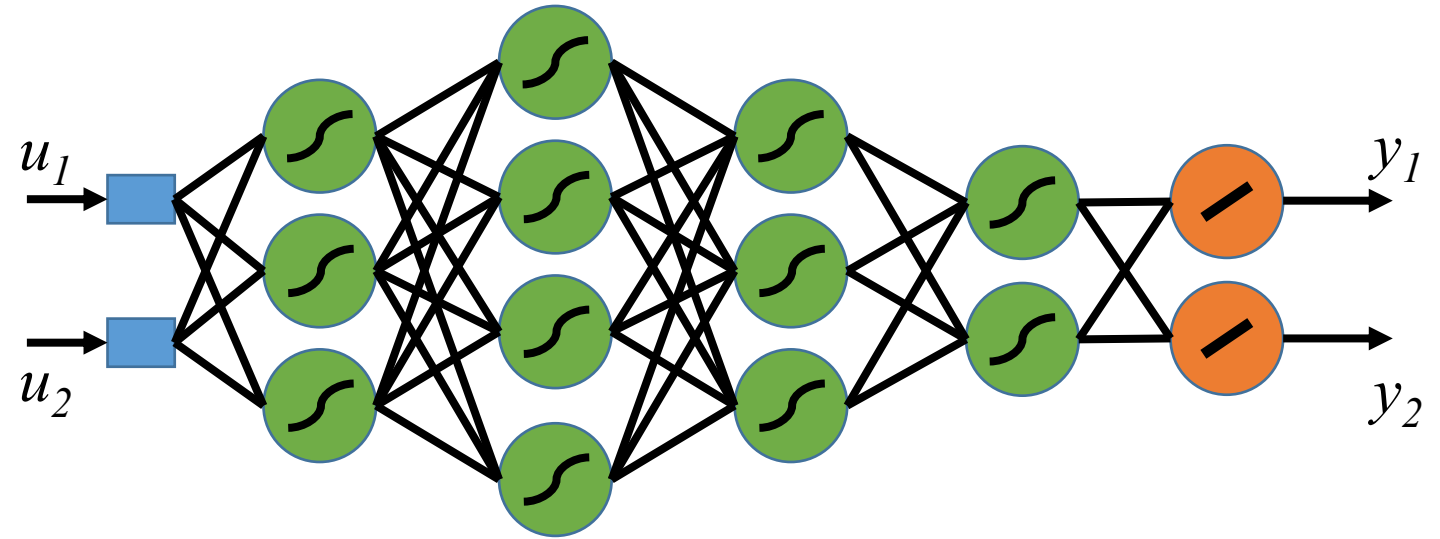
Data-Driven Modelling Process



Deep Learning

Larger Networks

Big Data



➔ Difficult for Training: Computational Load
Vanishing Gradient
Overfitting

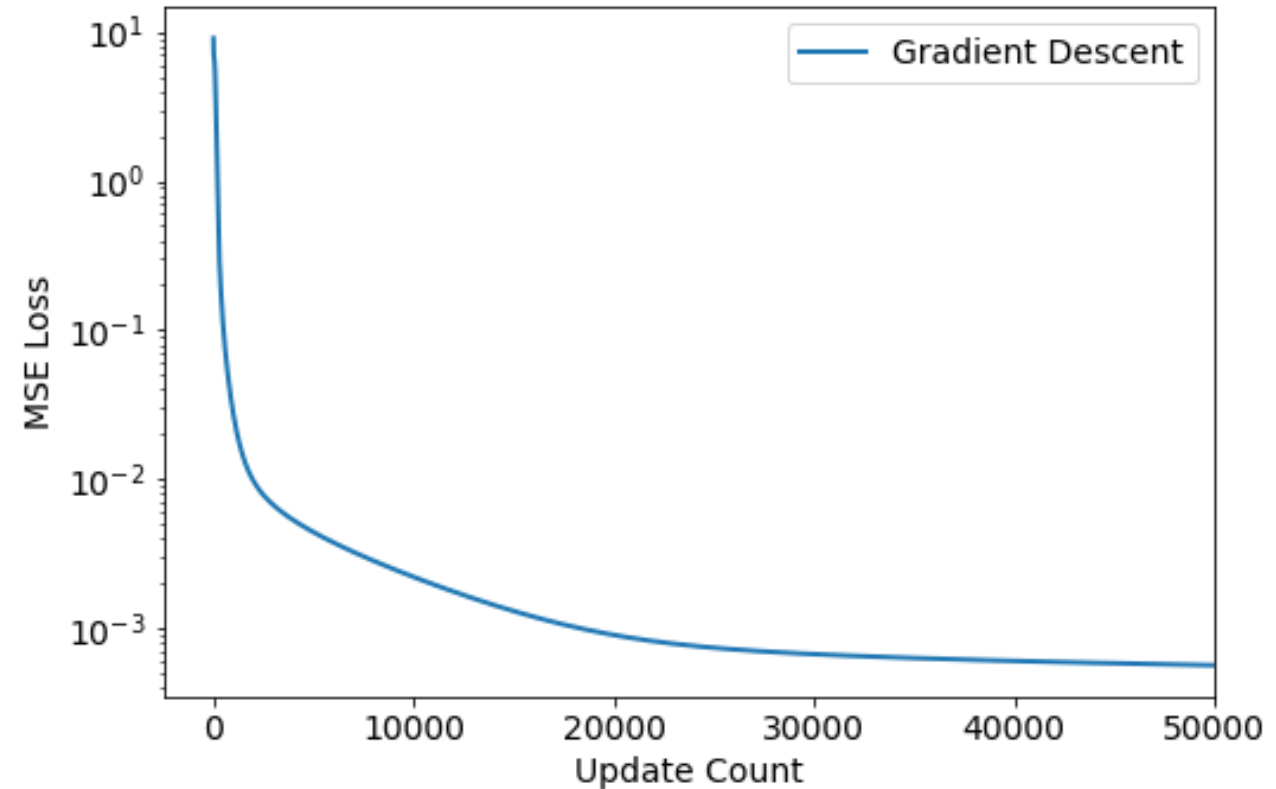
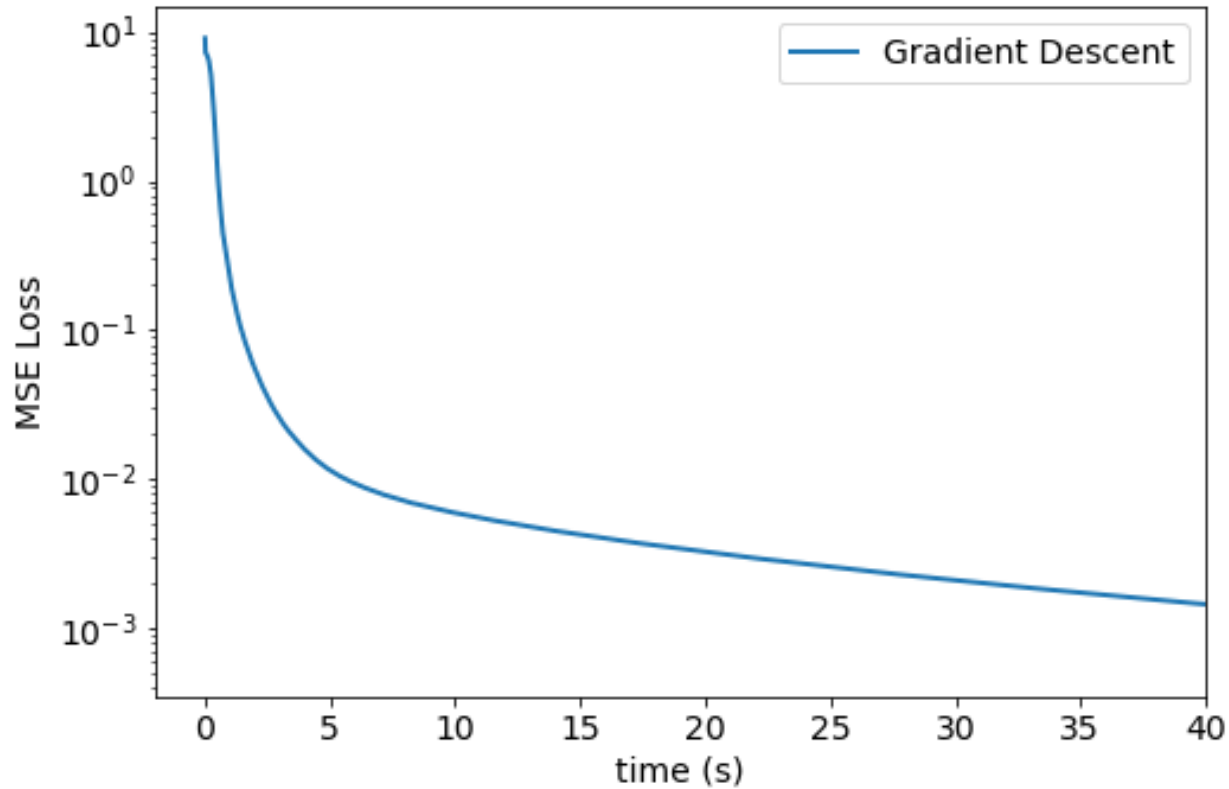
Stochastic Gradient Descent

Idea: Do we need to use the full dataset to compute the gradient?

➔ Use mini-batches of data to compute the gradient

This results in stochastic behavior as the mini-batch is only an approximation of the full dataset

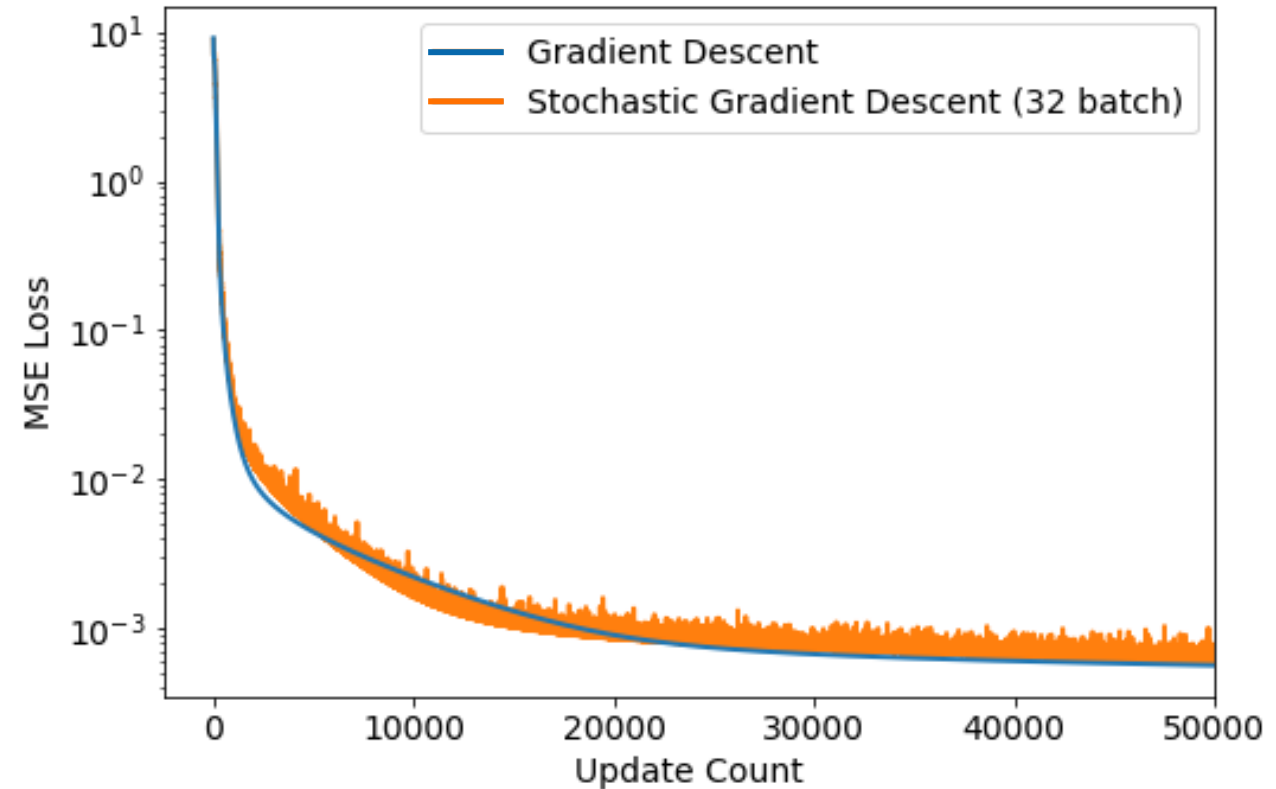
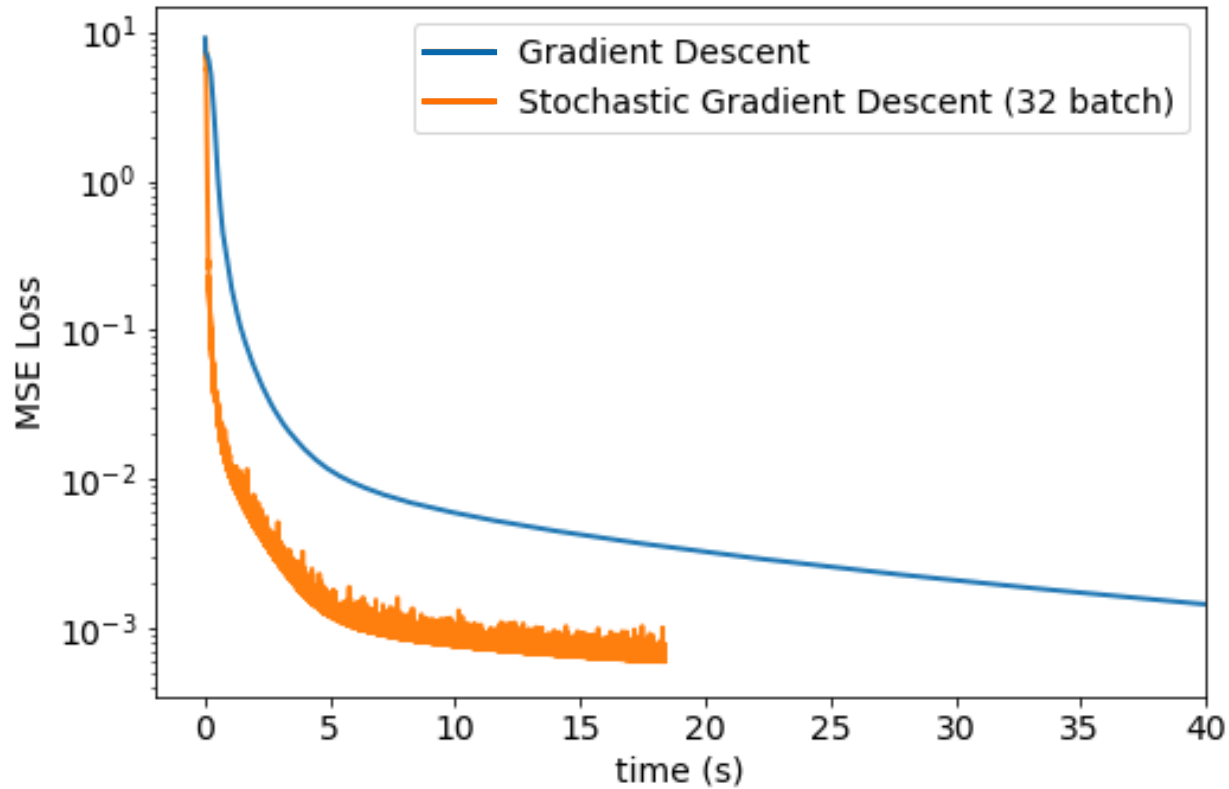
Stochastic Gradient Descent



$$y = u^2$$

10^4 samples, 1 hidden layer, 15 neurons (sigmoid activation), 1 white Gaussian input – std = 1,
1 (noisy) output, std noise = 0.02, learning rate 0.05

Stochastic Gradient Descent



$$y = u^2$$

10^4 samples, 1 hidden layer, 15 neurons (sigmoid activation), 1 white Gaussian input – std = 1,
1 (noisy) output, std noise = 0.02, learning rate 0.05, mini-batch size = 32

Stochastic Gradient Descent - Convergence

If cost-function is (locally) convex, differentiable and Lipschitz-continuous gradient + diminishing learning rate

Then GD and SGD converge to the closest (local) minimum

GD: $f(x^{(n)}) - f^* = \mathcal{O}\left(\frac{1}{n}\right)$

SGD: $f(x^{(n)}) - f^* = \mathcal{O}\left(\frac{1}{\sqrt{n}}\right)$ (batch size = 1)

➔ SGD has (much) slower theoretical convergence

Stochastic Gradient Descent - Convergence

If cost-function is (locally) convex, differentiable and Lipschitz-continuous gradient + diminishing learning rate

Then GD and SGD converge to the closest (local) minimum

GD: $f(x^{(n)}) - f^* = \mathcal{O}\left(\frac{1}{n}\right)$

SGD: $f(x^{(n)}) - f^* = \mathcal{O}\left(\frac{1}{\sqrt{n}}\right)$ (batch size = 1)

In practice: batch SGD, with fixed learning rate

reduce in computational cost / iteration outweigh disadvantages

Stochastic Gradient Descent - Extensions

Momentum

Next parameter update is linear combination of current gradient and previous updates

Adaptive Gradient (AdaGrad)

Adaptive learning rate per parameter

Adaptive Moment estimation (AdaM)

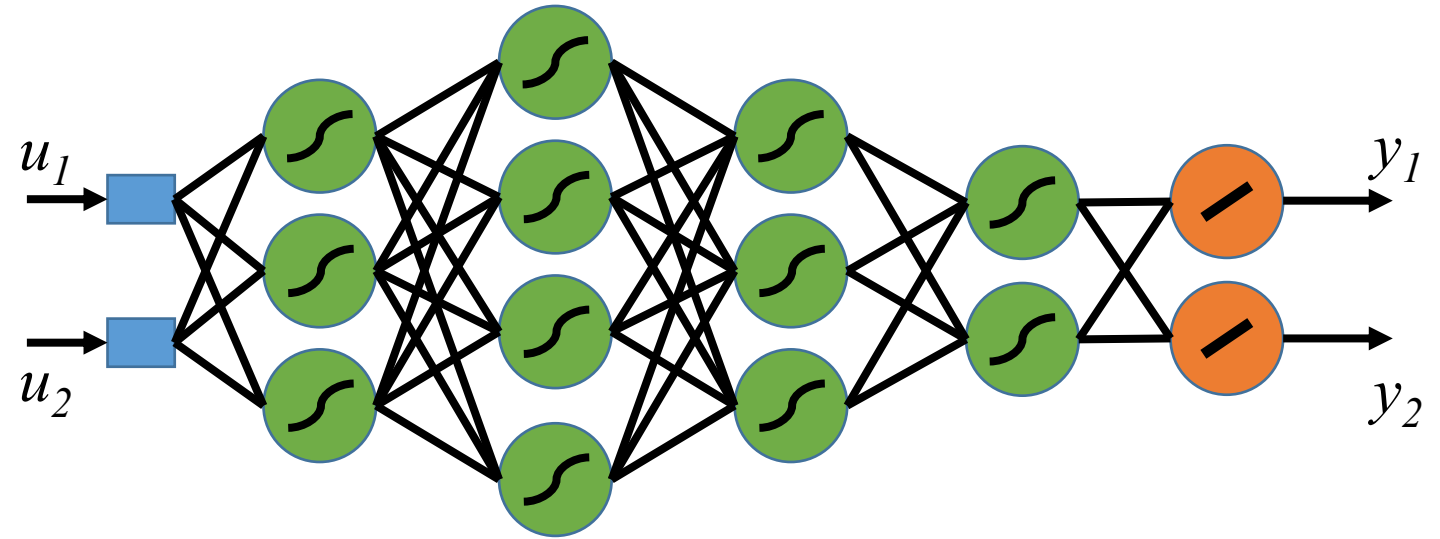
Uses a running average of the gradient and second moment of the gradient

....

Deep Learning

Larger Networks

Big Data



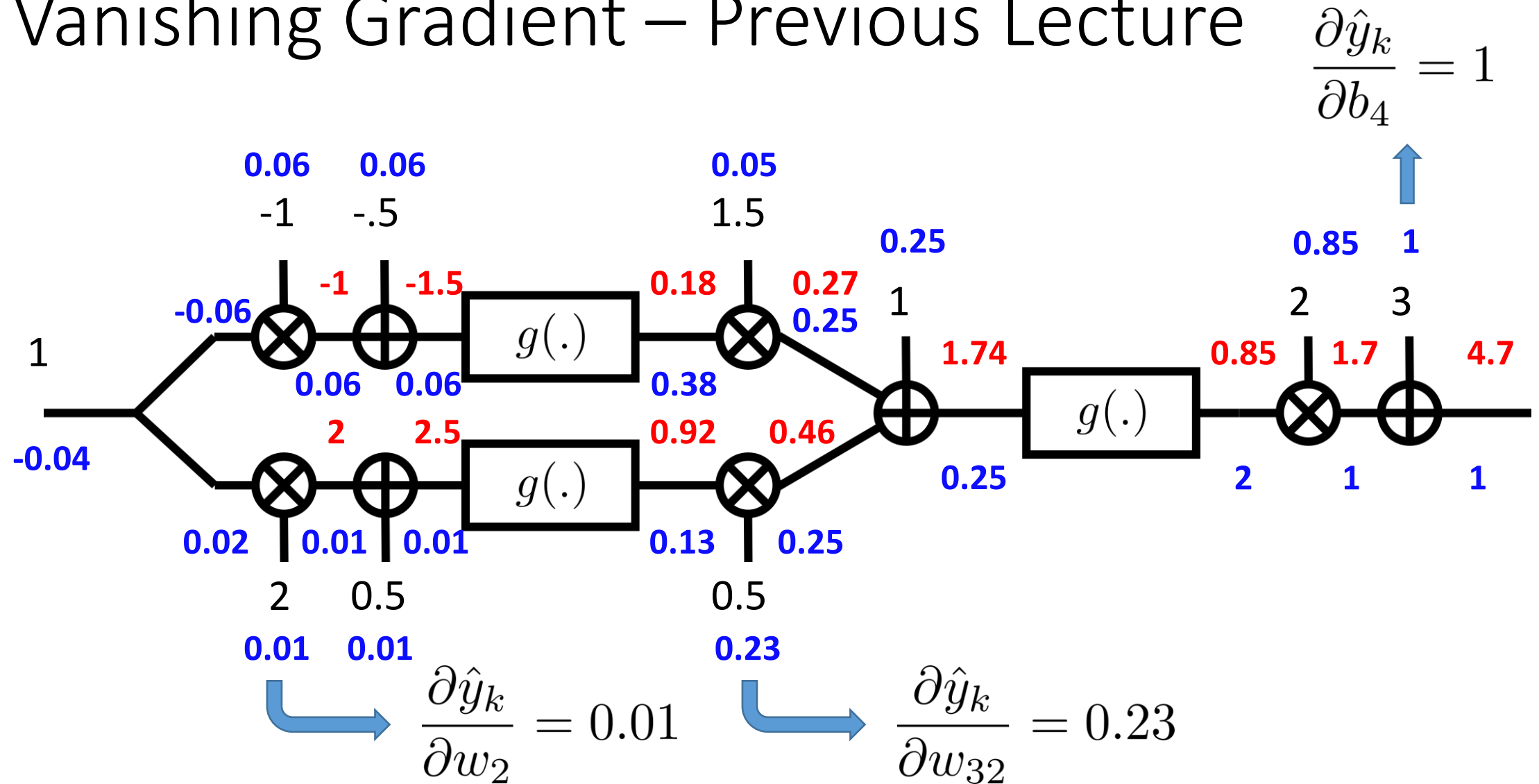
➔ Difficult for Training: Computational Load
Vanishing Gradient
Overfitting

Vanishing Gradient

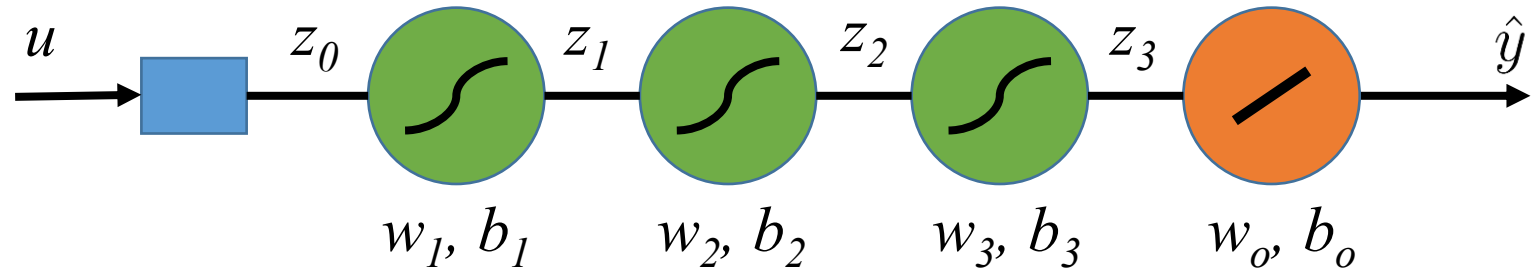
The gradient information becomes vanishing small when backpropagating backwards through the deep neural network

Prevents effective weight and bias training

Vanishing Gradient – Previous Lecture



Vanishing Gradient

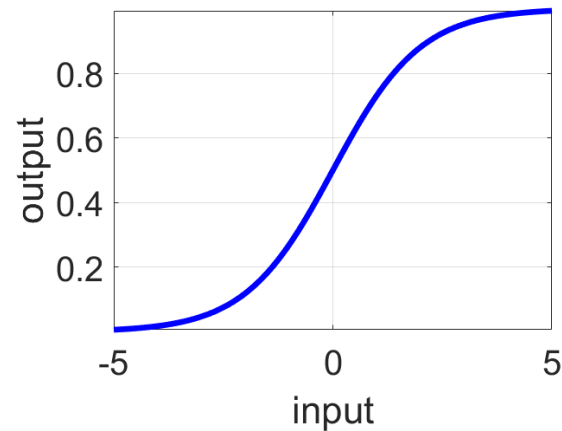


Typical activation function: sigmoid

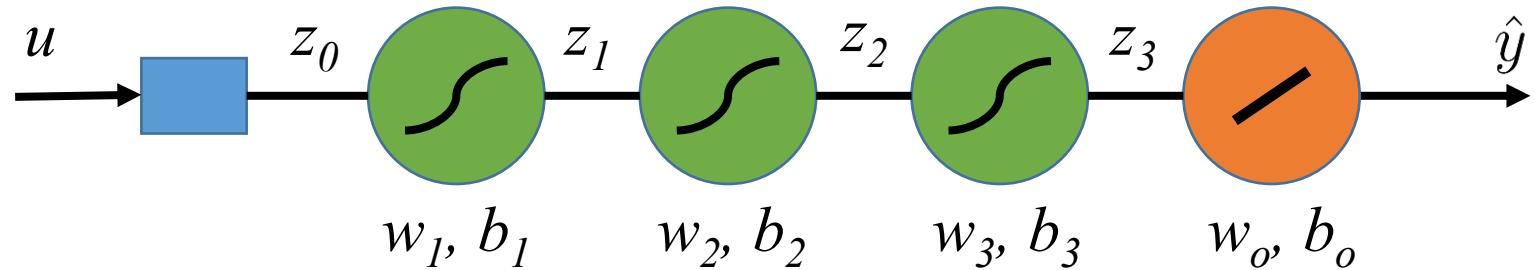
Initialization approach: (small) random weights

Sigmoid

$$g(x) = \frac{1}{1 + e^{-x}}$$



Vanishing Gradient

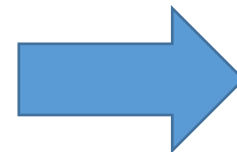
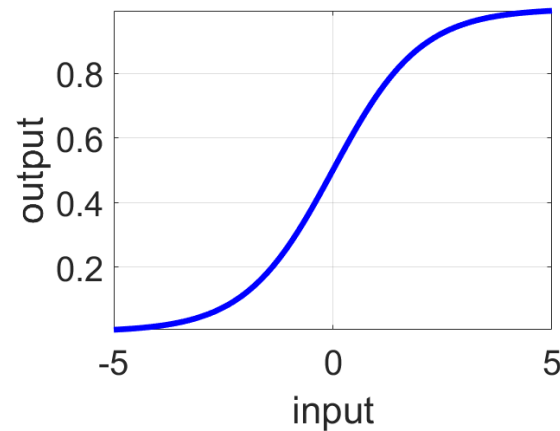


Typical activation function: sigmoid

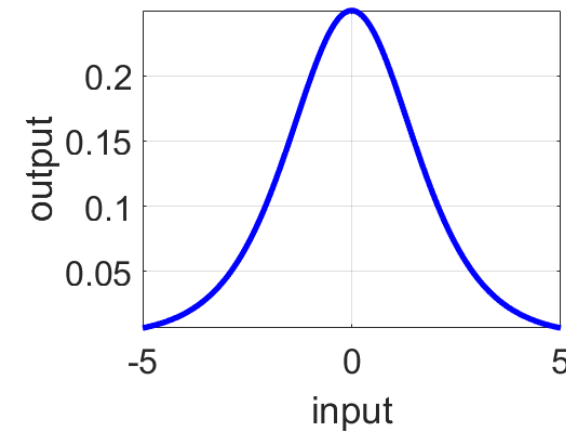
Initialization approach: (small) random weights

Sigmoid

$$g(x) = \frac{1}{1 + e^{-x}}$$

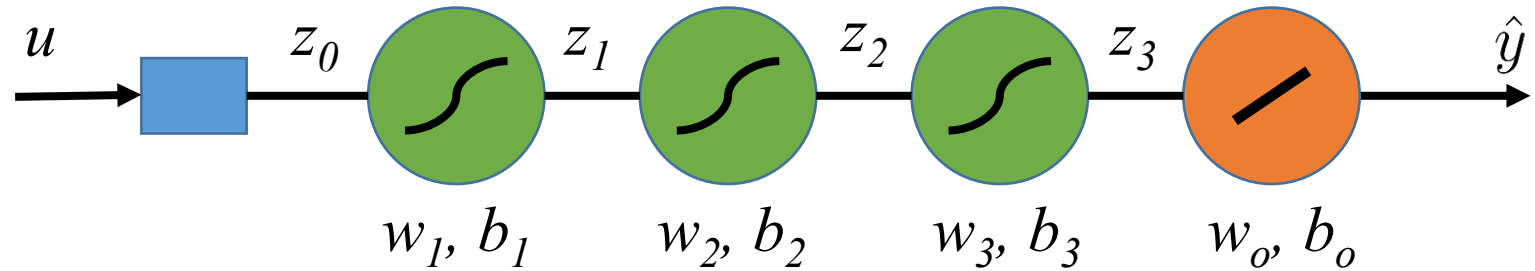


derivative



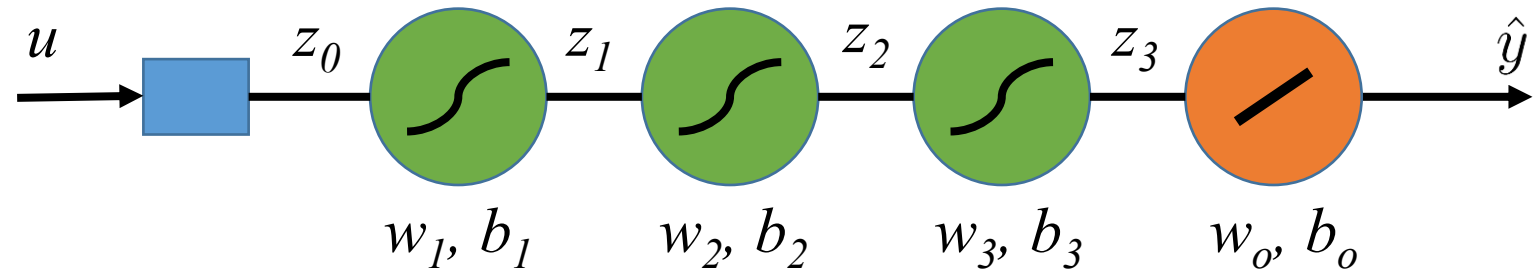
Max. of 0.25

Vanishing Gradient



$$\frac{\partial \hat{y}}{\partial b_1} = w_o \left. \frac{\partial g(x)}{\partial x} \right|_{x_3} w_3 \left. \frac{\partial g(x)}{\partial x} \right|_{x_2} w_2 \left. \frac{\partial g(x)}{\partial x} \right|_{x_1}$$

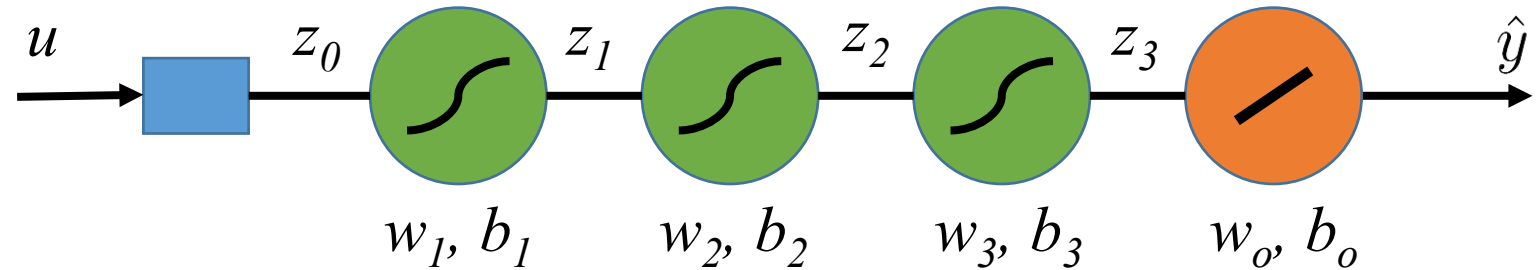
Vanishing Gradient



$$\frac{\partial \hat{y}}{\partial b_1} = \underbrace{w_o \frac{\partial g(x)}{\partial x} \Big|_{x_3}}_{\leq 0.25} \underbrace{w_3 \frac{\partial g(x)}{\partial x} \Big|_{x_2}}_{\leq 0.25} \underbrace{w_2 \frac{\partial g(x)}{\partial x} \Big|_{x_1}}_{\leq 0.25}$$

↓
Initialized to small values

Vanishing Gradient

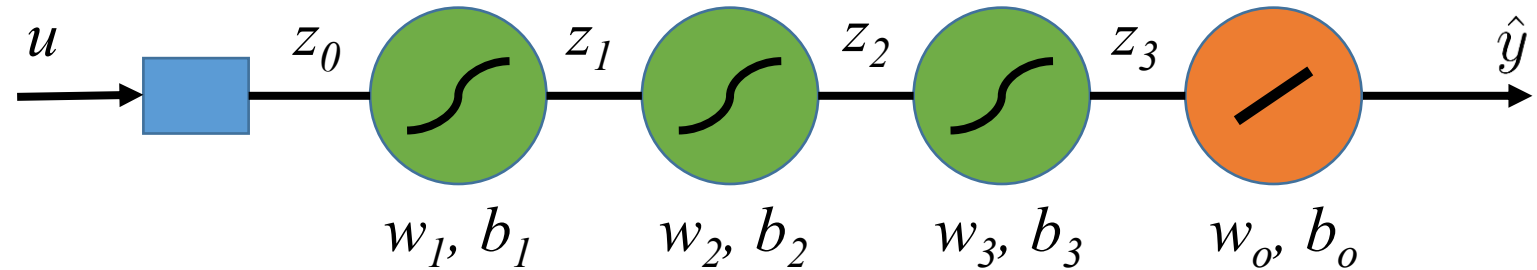


Gradient typically vanishes the further you propagate backwards

$$\frac{\partial \hat{y}}{\partial b_1} = \underbrace{w_o \frac{\partial g(x)}{\partial x} \Big|_{x_3}}_{\leq 0.25} \underbrace{w_3 \frac{\partial g(x)}{\partial x} \Big|_{x_2}}_{\leq 0.25} \underbrace{w_2 \frac{\partial g(x)}{\partial x} \Big|_{x_1}}_{\leq 0.25}$$

↓
Initialized to small values

Exploding Gradient



Gradient can also explode if derivative x weights > 1

$$\frac{\partial \hat{y}}{\partial b_1} = w_o \left. \frac{\partial g(x)}{\partial x} \right|_{x_3} w_3 \left. \frac{\partial g(x)}{\partial x} \right|_{x_2} w_2 \left. \frac{\partial g(x)}{\partial x} \right|_{x_1}$$

Especially problematic for recurrent neural networks

Vanishing / Exploding Gradient: Solutions

Change activation function

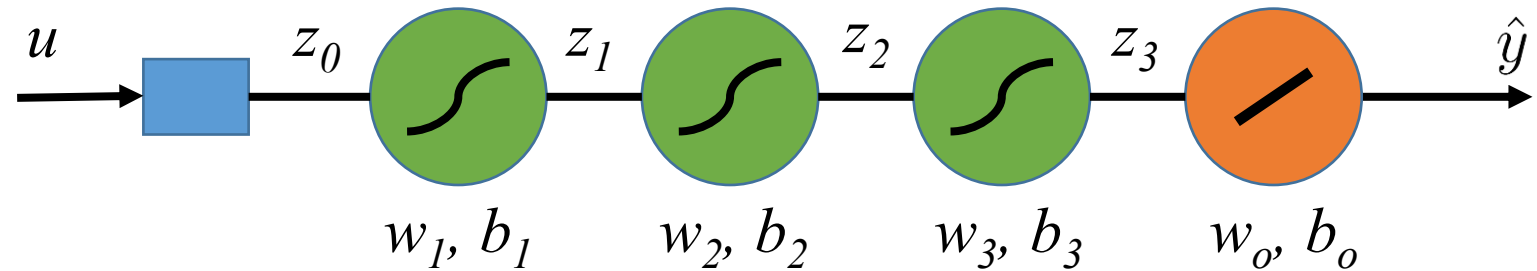
Data Normalization

Change network structure

Better Initialization

Faster Hardware

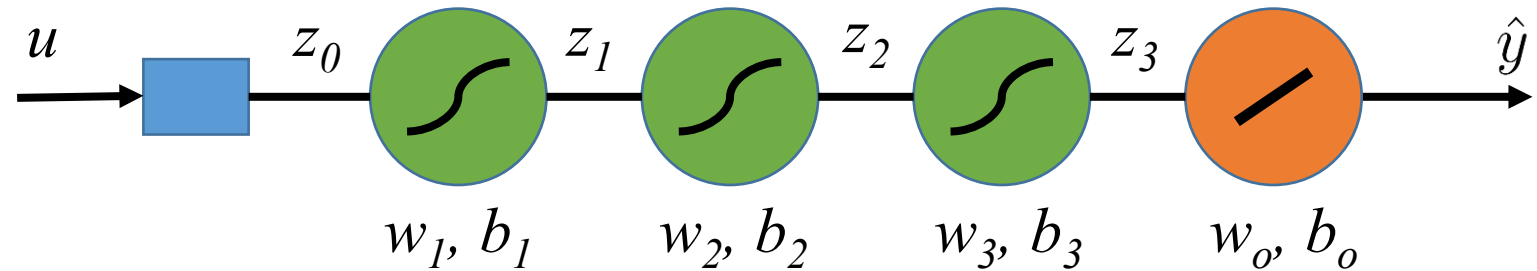
Changing the Activation Function



Gradient propagates well if derivative x weights ≈ 1

$$\frac{\partial \hat{y}}{\partial b_1} = w_o \left. \frac{\partial g(x)}{\partial x} \right|_{x_3} w_3 \left. \frac{\partial g(x)}{\partial x} \right|_{x_2} w_2 \left. \frac{\partial g(x)}{\partial x} \right|_{x_1}$$

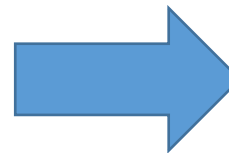
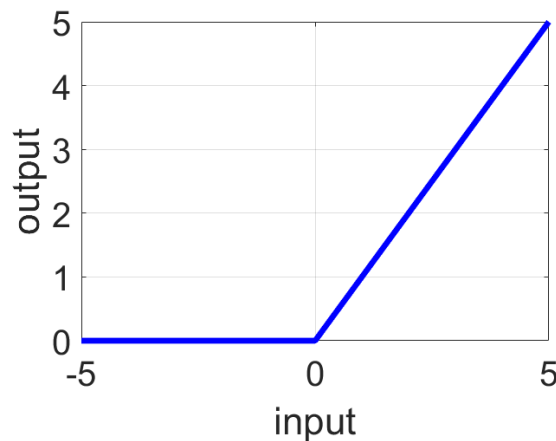
Changing the Activation Function



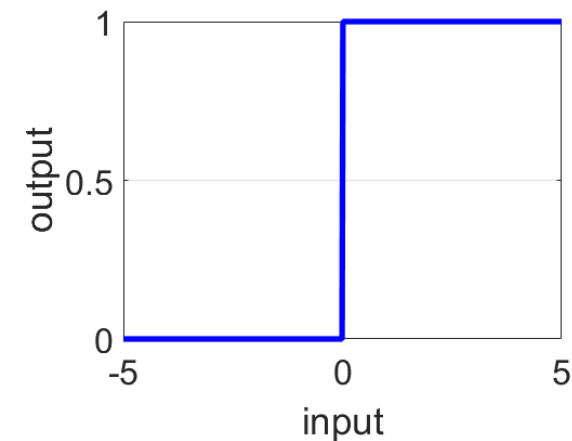
Gradient propagates well if derivative x weights ≈ 1

ReLu

$$g(x) = \begin{cases} 0 & \forall x < 0 \\ x & \forall x \geq 0 \end{cases}$$

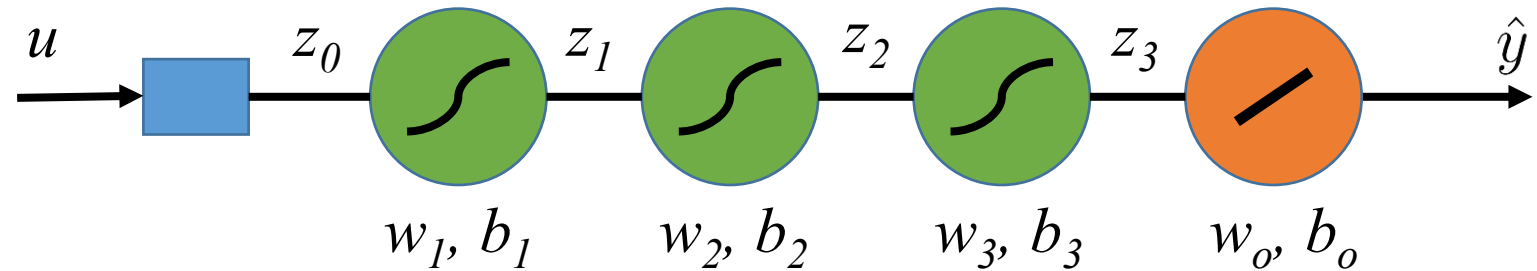


derivative



Max. of 1

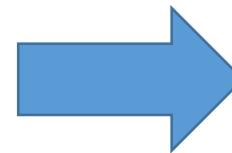
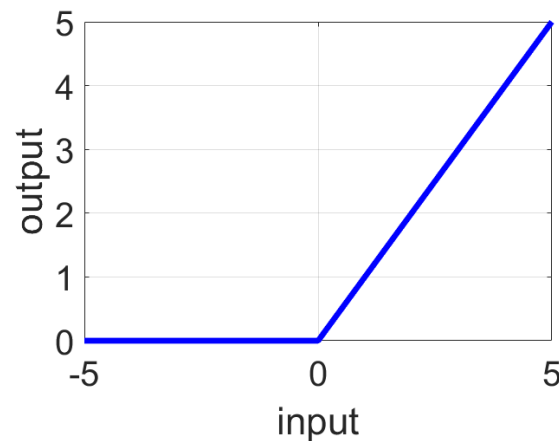
Changing the Activation Function



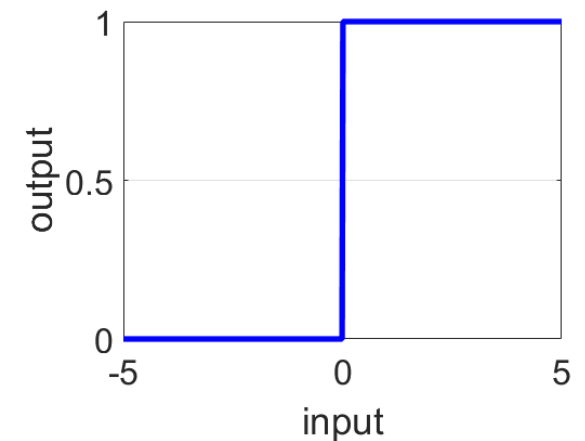
ReLU (and similar forms) became one of the most popular choices

ReLU

$$g(x) = \begin{cases} 0 & \forall x < 0 \\ x & \forall x \geq 0 \end{cases}$$



derivative



Vanishing / Exploding Gradient: Solutions

Change activation function

Data Normalization

Change network structure

Better Initialization

Faster Hardware

Data Normalization

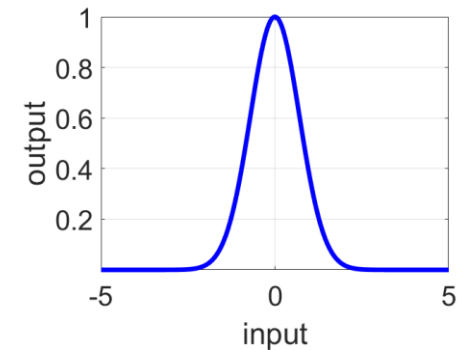
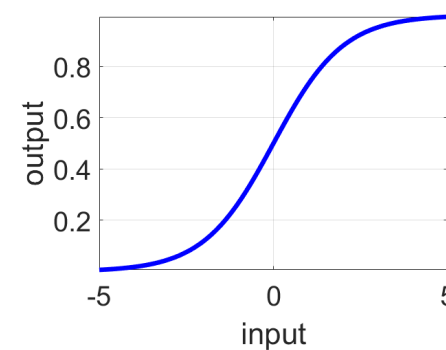
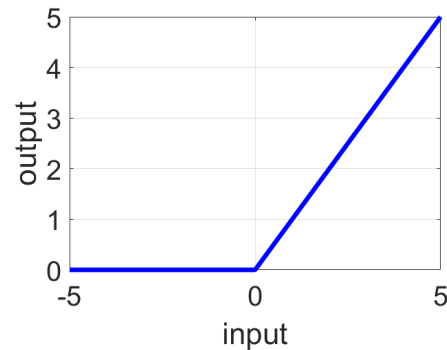
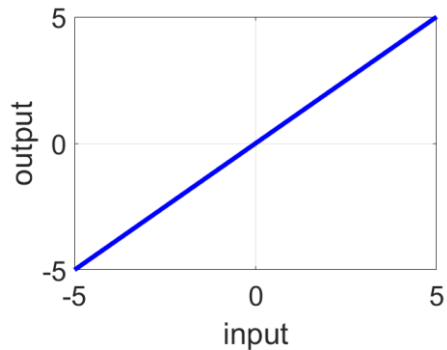
Always normalize the input-output data before training

zero-mean

unit variance

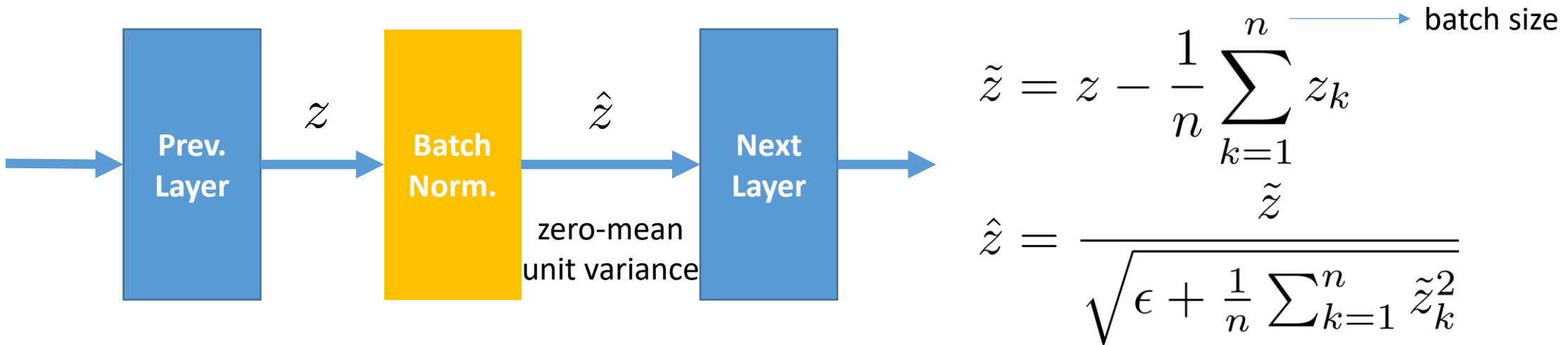
Improves the conditioning of the learning problem

Lowest risk of vanishing gradient problem since the full activation function range is used



Batch Normalization Layers

Add a normalization layer in the neural network architecture



First order statistics (mean, variance) are always the same, independent from the previous layers. Previous layers only affect the higher order statistics.

Vanishing / Exploding Gradient: Solutions

Change activation function

Data Normalization

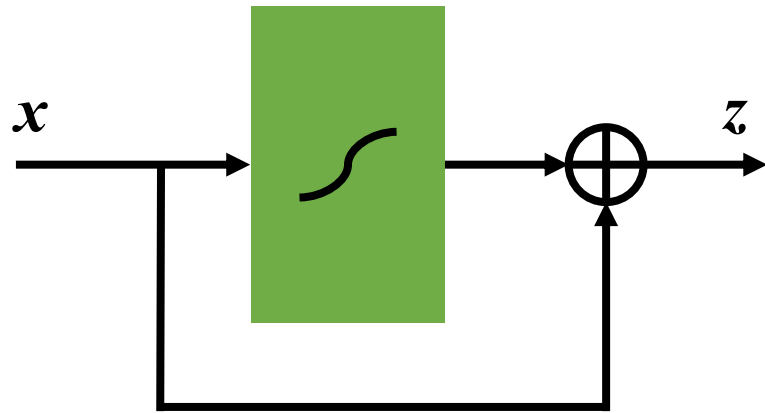
Change network structure

Better Initialization

Faster Hardware

Change Network Structure

Smart network structure allow for a better backpropagation of the gradient



Residual Network Layer

$$z = x + g(W_1x + b_1)$$

$$z = f(x) = x + g(W_1x + b_1)$$

$$\frac{\partial f(x)}{\partial x} = 1 + \frac{\partial g(x)}{\partial x}$$



Ensures values close to 1

Vanishing / Exploding Gradient: Solutions

Change activation function

Data Normalization

Change network structure

Better Initialization

Faster Hardware

Better Initialization

Vanishing Gradients → Slower Learning

- I. Better initialization leads you faster to the (local) optimum. The learning time is reduced.
- II. A smart initialization can avoid regions of vanishing / exploding gradients.

This only overcomes the vanishing gradient problem; it doesn't solve the root cause of it.

Vanishing / Exploding Gradient: Solutions

Change activation function

Data Normalization

Change network structure

Better Initialization

Faster Hardware

Faster Hardware

Vanishing Gradients → Slower Learning

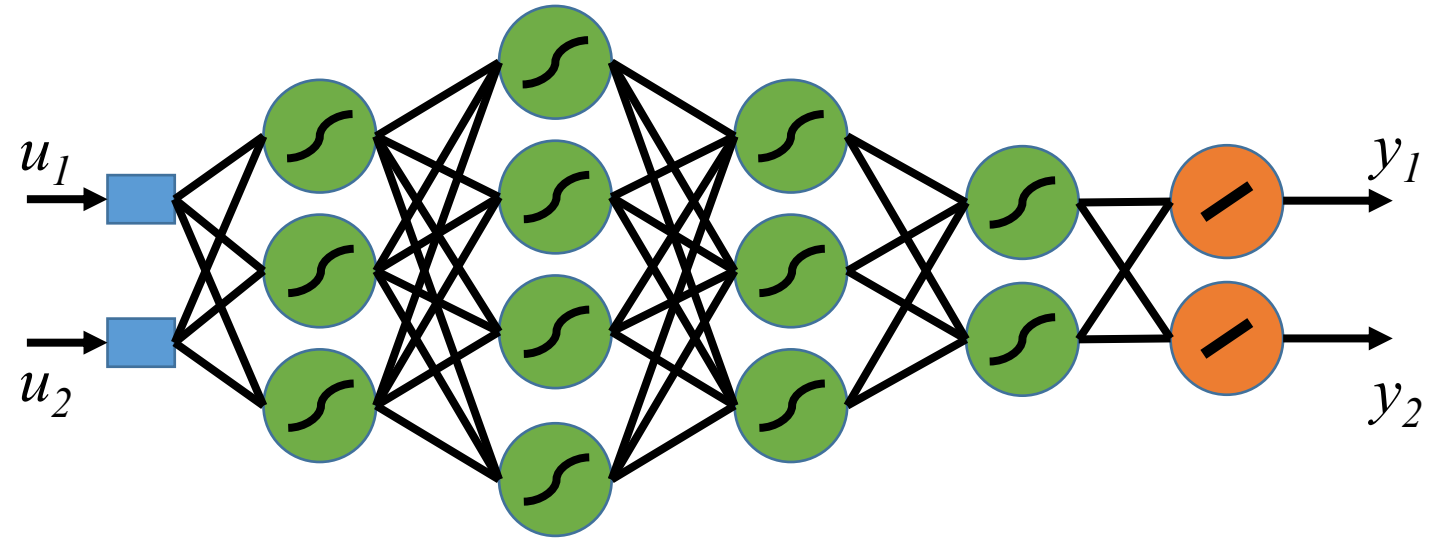
The development of faster and parallelized hardware has been one of the major drivers in the development of deep learning.

This only overcomes the vanishing gradient problem; it doesn't solve the root cause of it.

Deep Learning

Larger Networks

Big Data



➔ Difficult for Training: Computational Load
Vanishing Gradient
Overfitting

Overfitting & Regularization

Parameter Norm Regularization (previous lecture)

Early Stopping (previous lecture)

Parameter Sharing

Data Augmentation

Noise Robustness

Sparse Representations

Parameter Sharing

Share parameters over multiply neurons / layers

Often used in Convolutional Networks or multi-task learning problems

Convolutional Operation

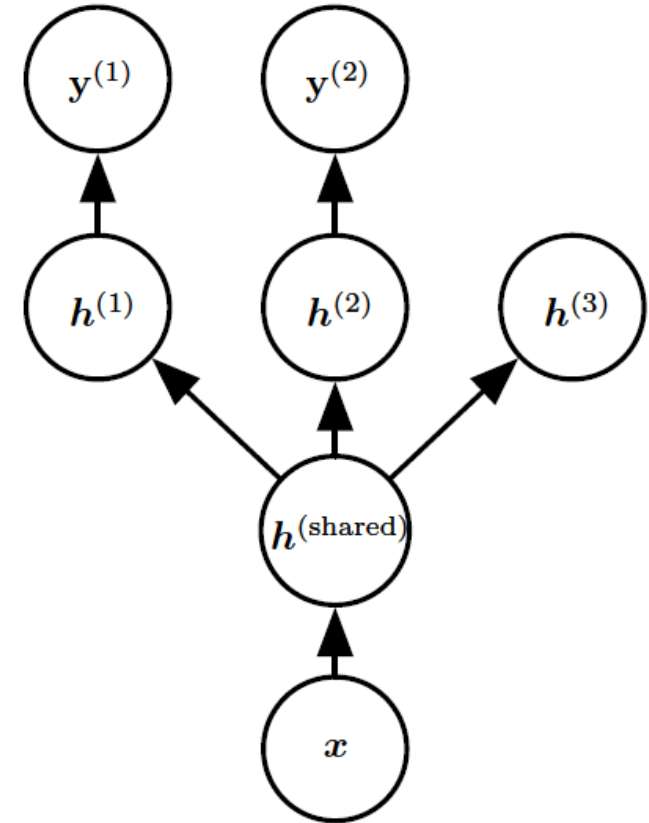
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature

source: <https://towardsdatascience.com/>



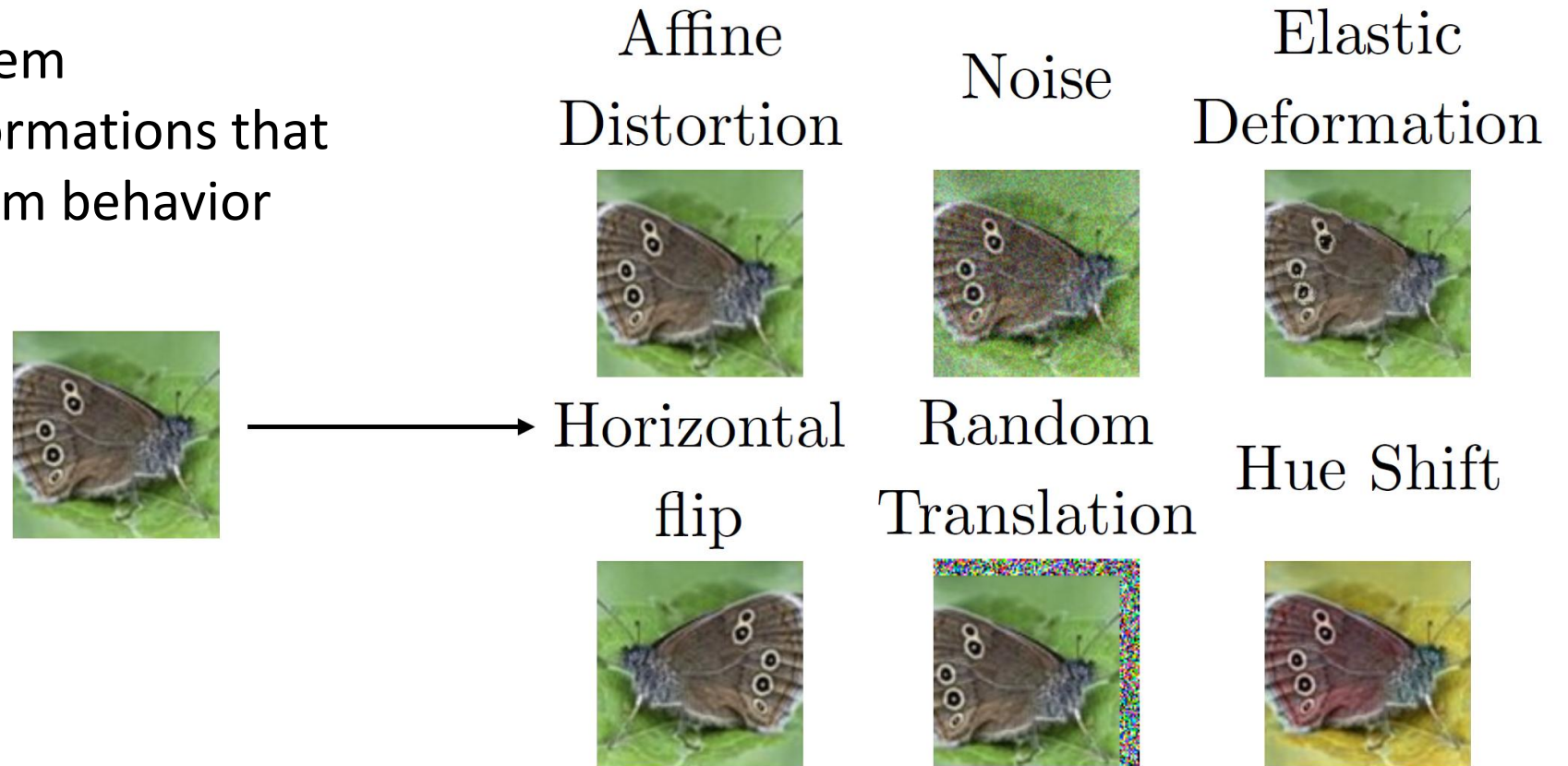
Multi-Task Learning

e.g. multiple systems / outputs with shared dynamics

source: www.deeplearningbook.org

Data Augmentation

Make use of data / system symmetries and transformations that do not impact the system behavior



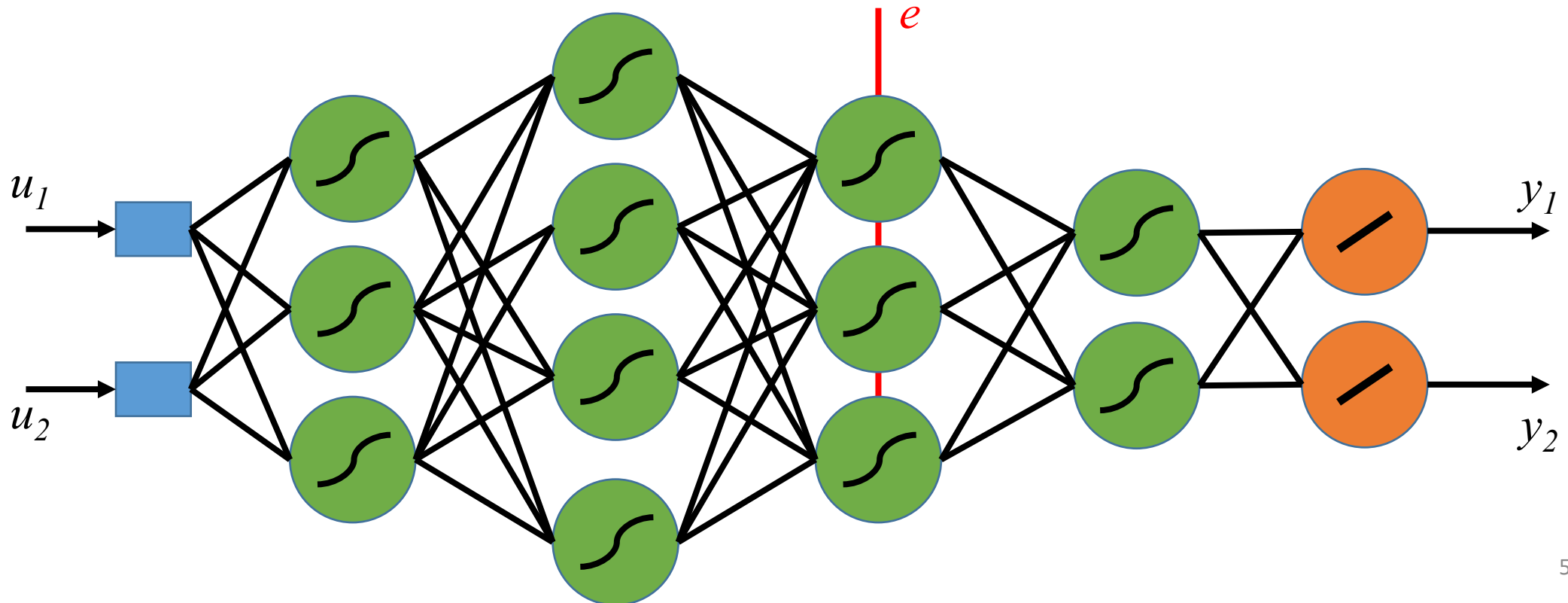
source: www.deeplearningbook.org

Noise Robustness

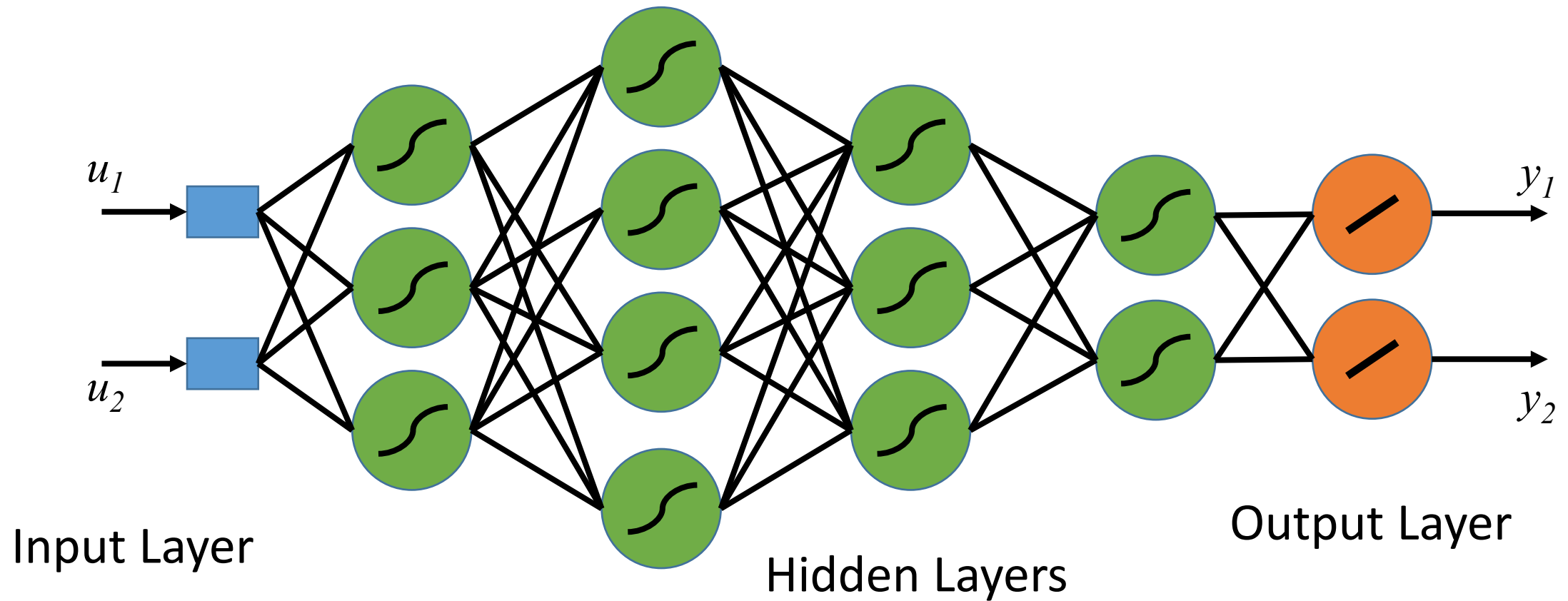
Add noise to the (hidden) layers

Increases robustness at cost of bias introduction

Links with L^2 -regularization of parameters

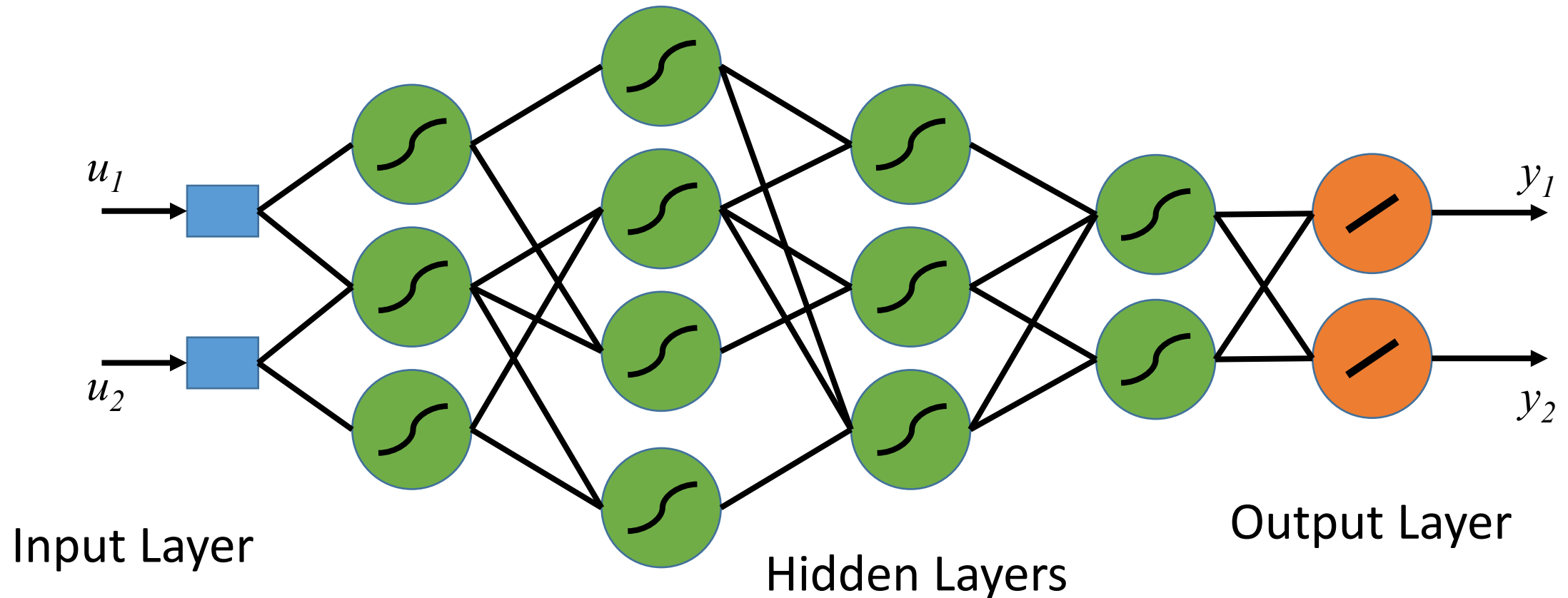


Sparse Connections



Sparse Connections

Reduces the number of weights to be trained
e.g. L^1 -norm regularization of the weights



Sparse Representations

Instead of having sparse weights, obtain a sparse representation (i.e. sparse signals)
e.g. L¹-norm regularization of the representation

$$\begin{array}{ccc} \begin{bmatrix} 18 \\ 5 \\ 15 \\ -9 \\ -3 \end{bmatrix} & = & \begin{bmatrix} 4 & 0 & 0 & -2 & 0 & 0 \\ 0 & 0 & -1 & 0 & 3 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 & -4 \\ 1 & 0 & 0 & 0 & -5 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ -2 \\ -5 \\ 1 \\ 4 \end{bmatrix} \\ \mathbf{y} \in \mathbb{R}^m & & \mathbf{A} \in \mathbb{R}^{m \times n} \quad \mathbf{x} \in \mathbb{R}^n \end{array}$$

sparse weights / connections

$$\begin{array}{ccc} \begin{bmatrix} -14 \\ 1 \\ 19 \\ 2 \\ 23 \end{bmatrix} & = & \begin{bmatrix} 3 & -1 & 2 & -5 & 4 & 1 \\ 4 & 2 & -3 & -1 & 1 & 3 \\ -1 & 5 & 4 & 2 & -3 & -2 \\ 3 & 1 & 2 & -3 & 0 & -3 \\ -5 & 4 & -2 & 2 & -5 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \\ 0 \\ 0 \\ -3 \\ 0 \end{bmatrix} \\ \mathbf{y} \in \mathbb{R}^m & & \mathbf{B} \in \mathbb{R}^{m \times n} \quad \mathbf{h} \in \mathbb{R}^n \end{array}$$

sparse representation

Many other methods

Bagging

model output = mean of an ensemble of trained network structures

Dropout

optimize over an ensemble of network structures

Adversarial Training

training on adversarial perturbed examples from the training set

Artificial Neural Networks

Deep Learning & Deep Neural Networks

Training a Deep Neural Network

Artificial Neural Networks for Dynamical Systems

Model Structures

NARX

NOE

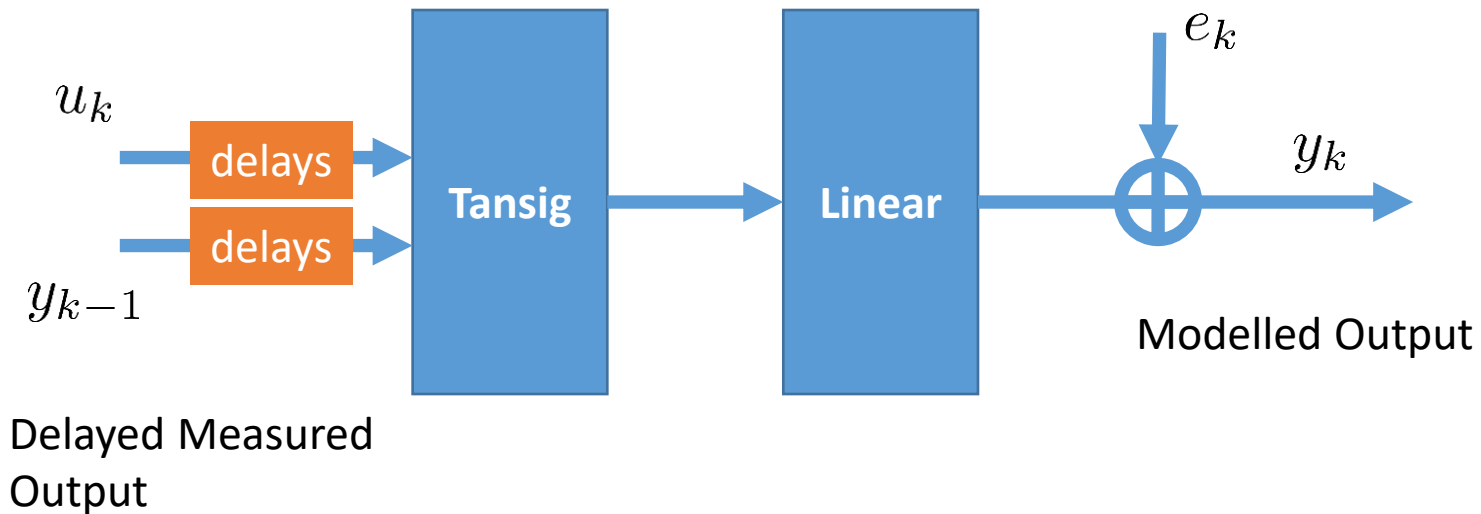
Recurrent ANN

Recursive State-Space

Unwrapped State-Space

Feedforward NN - NARX

Feedforward NN with delayed inputs and delayed measured outputs (NARX)

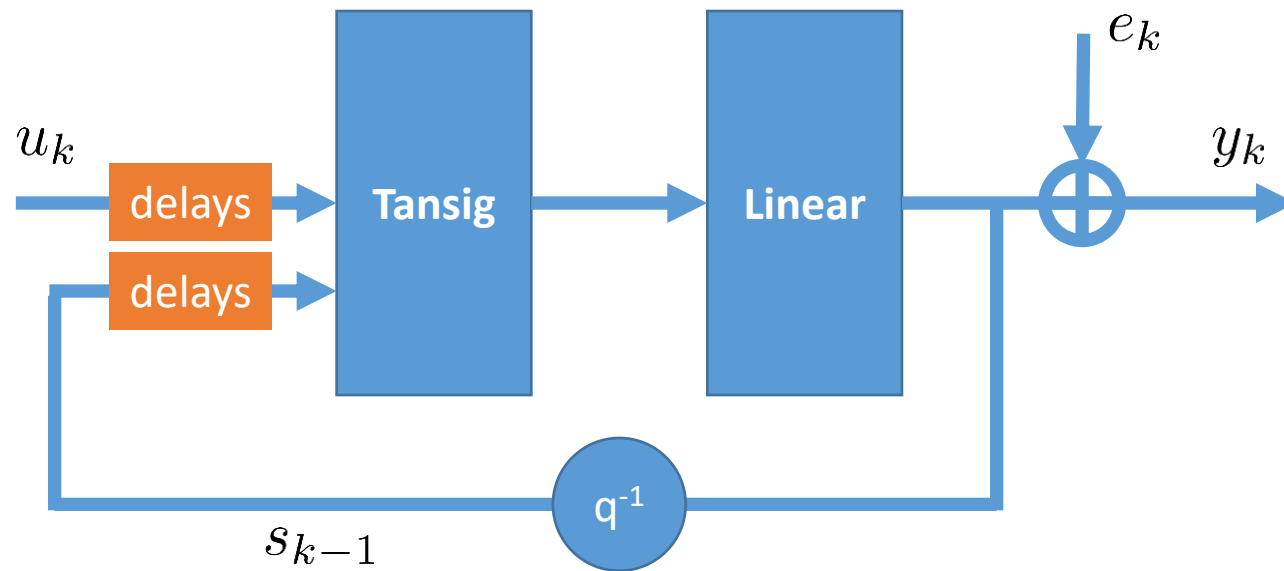


**Nonlinear Auto Regressive with
eXogeneous Input (NARX)**

Very particular noise structure
Not always easy to analyze

$$y_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, y_{k-1}, \dots, y_{k-n_a}) + e_k$$

Recurrent NN - NOE



Loop output back

Nonlinear Output Error (NOE)

Similar to NARX, but different noise handling

Output depends on past unknown noiseless outputs

Difficult to analyze

Difficult to estimate

Straightforward noise structure

$$s_k = f(u_k, u_{k-1}, \dots, u_{k-n_b}, s_{k-1}, \dots, s_{k-n_a})$$

$$y_k = s_k + e_k$$

Intermezzo: 1-Step Ahead Prediction

$$\hat{y}_k = \mathbb{E} \{ y_k | \mathcal{D}_{k-1}, u_k \}$$

data up to timestep $k-1$

predicted output
at timestep k

input at timestep k

Expectation taken over the stochastic disturbances

Intermezzo: Simulation

$$\hat{y}_k = \mathbb{E} \left\{ y_k \mid \{u_j\}_{j=1}^k \right\}$$

↓
simulated output
at timestep k

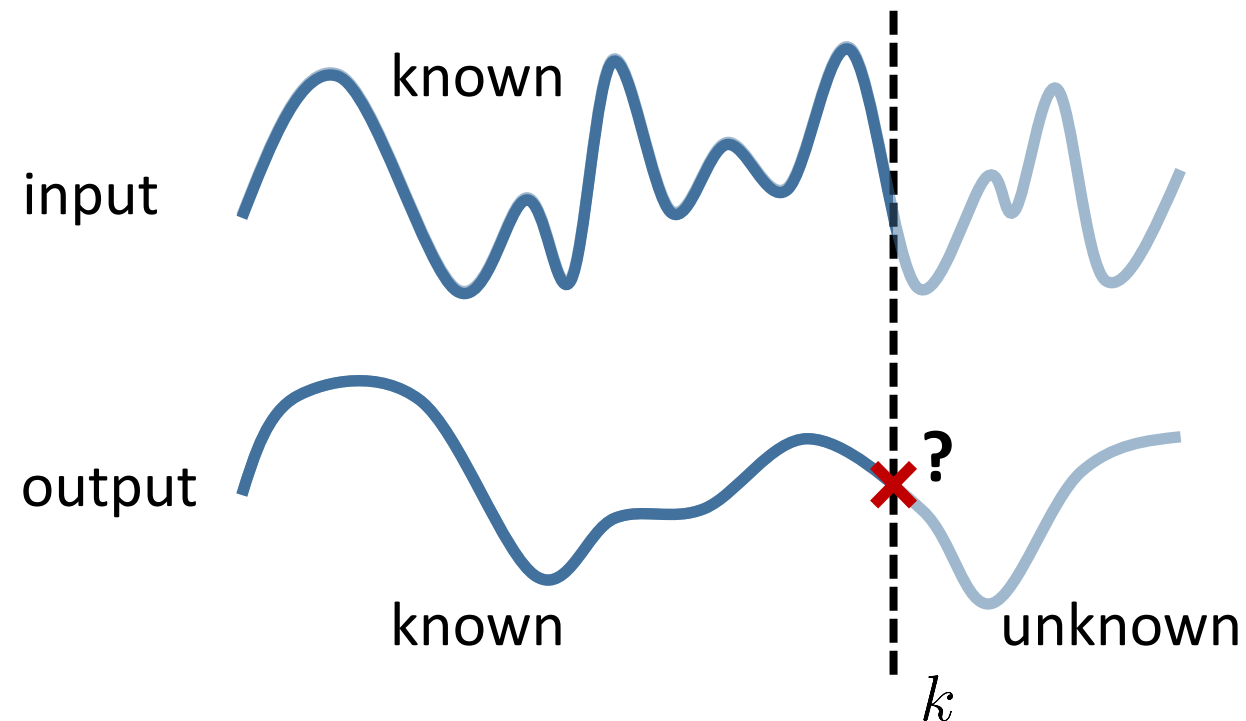
↓
inputs from start
up to timestep k

Expectation taken over the stochastic disturbances

Intermezzo: Prediction vs Simulation

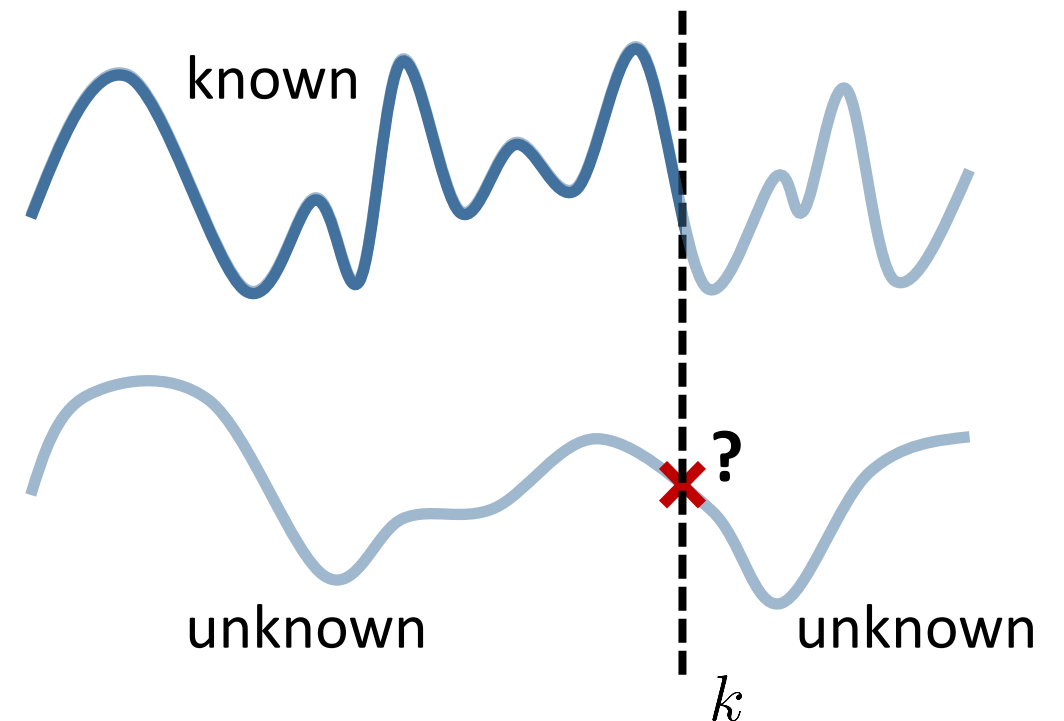
Prediction

$$\hat{y}_k = \mathbb{E} \{y_k | \mathcal{D}_{k-1}, u_k\}$$



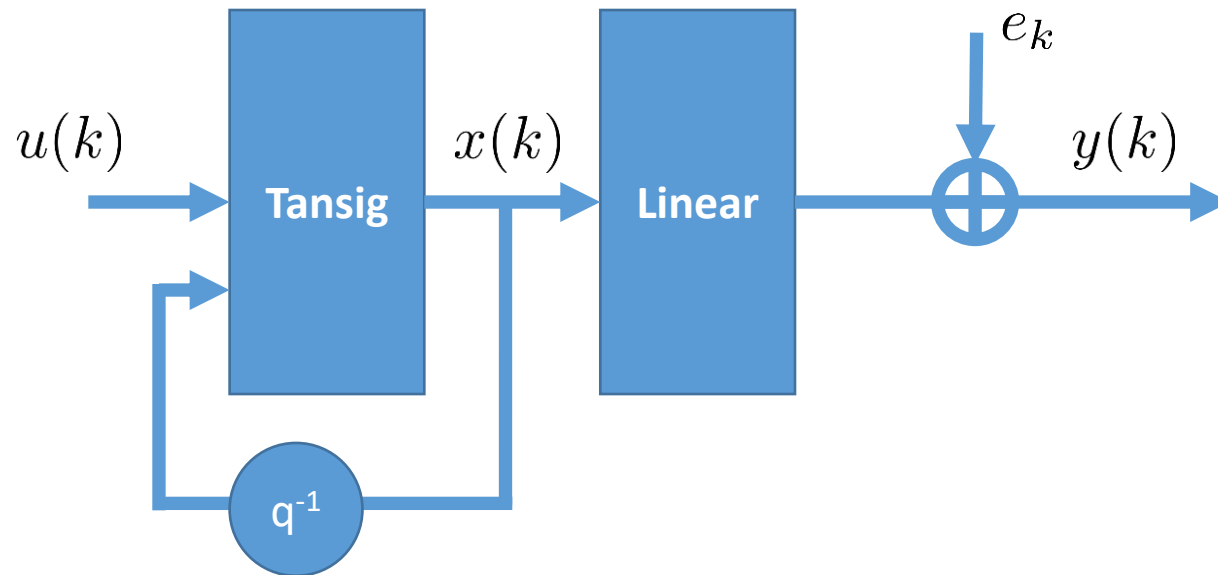
Simulation

$$\hat{y}_k = \mathbb{E} \{y_k | \{u_j\}_{j=1}^k\}$$



Recurrent NN – State-Space

Recurrent NN can be interpreted as a state-space representation



Loop over one layer

$$x_k = f(x_{k-1}, u_k)$$

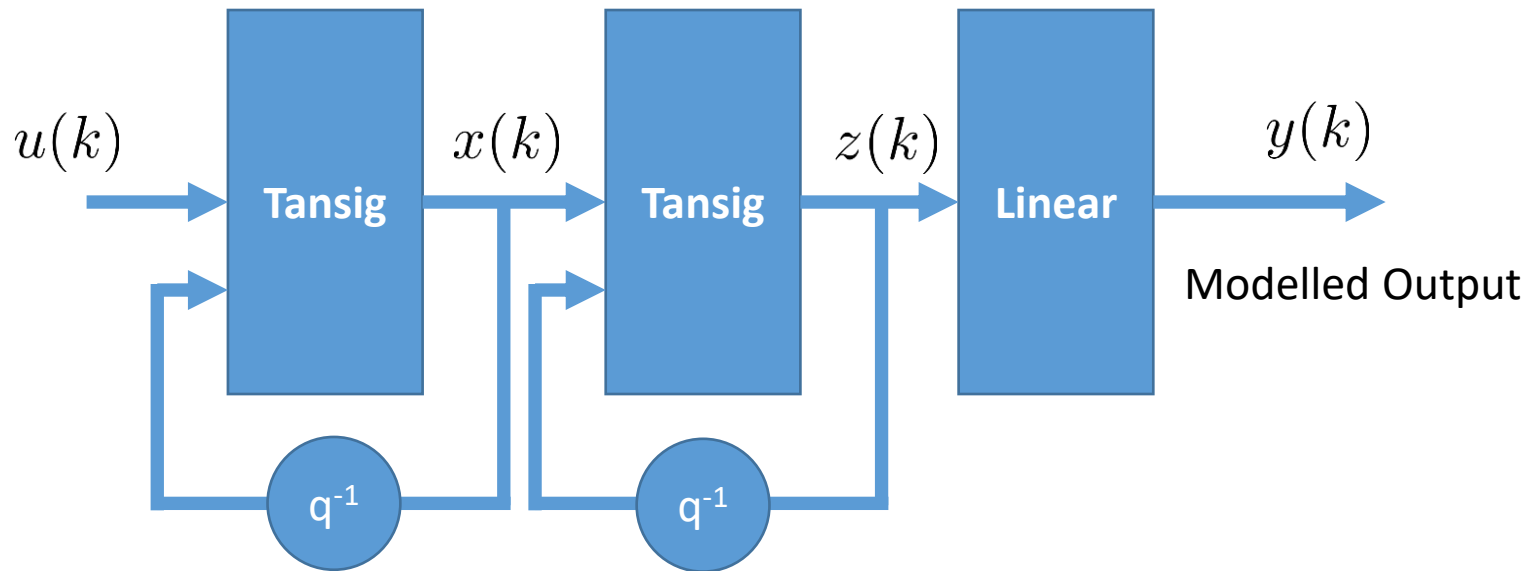
$$y_k = Cx_k$$

$x(k)$ are states of the model

As many states as we have
neurons in the hidden layer

Recurrent NN – State-Space

Recurrent NN can be interpreted as a state-space representation



Loop over one layer

$$x_k = f(x_{k-1}, u_k)$$

$$z_k = g(z_{k-1}, x_k)$$

$$y_k = C z_k$$



$$x_k = f(x_{k-1}, u_k)$$

$$z_k = g(z_{k-1}, f(x_{k-1}, u_k))$$

$$y_k = C z_k$$



$$\begin{bmatrix} x_k \\ z_k \end{bmatrix} = \tilde{f} \left(\begin{bmatrix} x_{k-1} \\ z_{k-1} \end{bmatrix}, u_k \right)$$

$$y_k = C z_k$$

$x(k), z(k)$ are states of the model

As many states as we have
neurons in the hidden layers

Recurrent NN – General Form

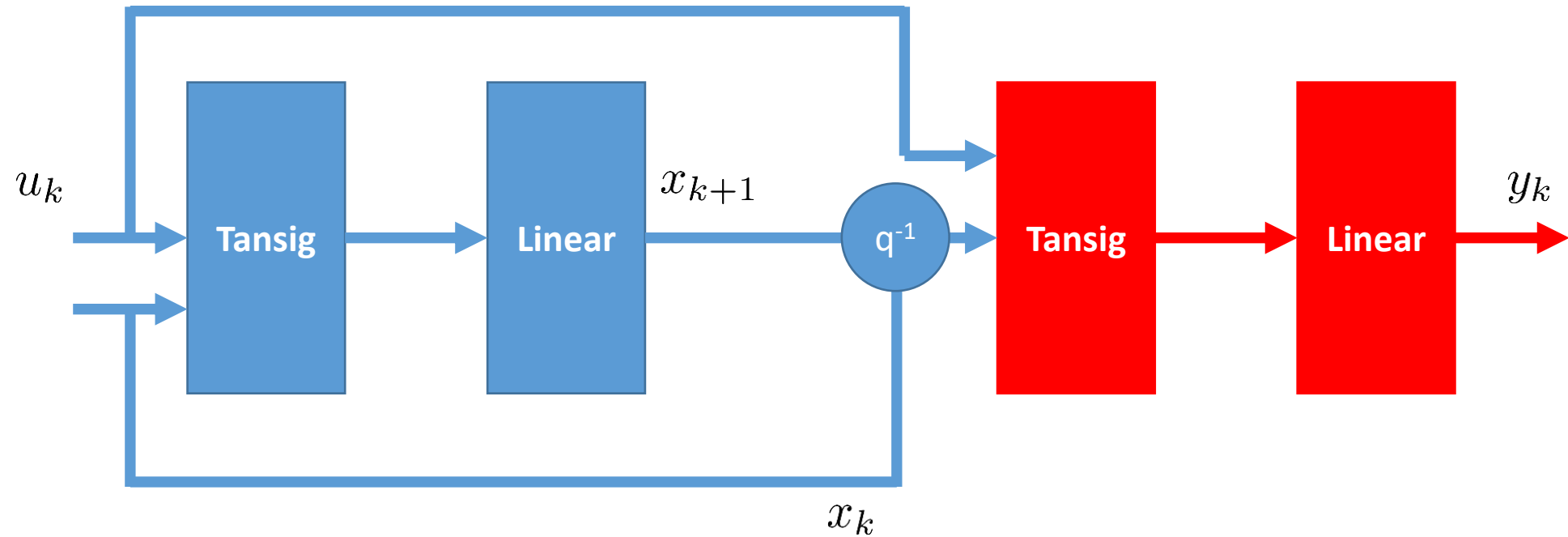
$$x_k = f(x_{k-1}, u_k, y_{k-1})$$
$$y_k = g(x_k)$$

Comprises a huge range of recurrent NN structures, including LSTM

How to structure f, g ? → Model structure selection / design

NN Training? → Initialization, training strategy,
expanded training network

State-Space Neural Network (SSNN)



$$x_{k+1} = f(x_k, u_k)$$

$$y_k = g(x_k, u_k)$$

State dimension = # neurons in the blue linear layer

Continuous Time Models

Continuous-Time ANN

ODENet: explicitly include integration schemes in the ANN

Hamiltonian / Lagrangian ANN: describe the Hamiltonian / Lagrangian as an ANN and explicitly include integration schemes (energy conservation)

Continuous Time Models

Continuous-Time ANN

$$\begin{aligned} \min_{\theta, x(0)} \quad & \frac{1}{N} \sum_{k=0}^{N-1} ||\hat{y}(k\Delta t) - y(k\Delta t)||_2^2 \\ \text{s. t.} \quad & \frac{dx(t)}{dt} = f_{\theta}(x(t), u(t)) \\ & \hat{y}(t) = h_{\theta}(x(t), u(t)) \end{aligned}$$

Integrator in the optimization?

Continuous Time Models

RK4 integration

$$x(\Delta t) = x(0) + \frac{1}{6} \Delta t (k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_4 = f(x(0) + \Delta t k_3, u(\Delta t))$$

$$k_3 = \dots$$

Sufficient:
Differentiable
operations

$$\bullet \frac{dx(\Delta t)}{dx(0)} = I + \frac{1}{6} \Delta t \left(\frac{dk_1}{dx(0)} + 2 \frac{dk_2}{dx(0)} + 2 \frac{dk_3}{dx(0)} + \frac{dk_4}{dx(0)} \right)$$

$$\bullet \frac{dk_4}{dx(0)} = \frac{df(x(0) + \Delta t k_3, u(0))}{dx(0)} = \frac{df}{dx} \Big|_{x=x(0) + \Delta t k_3} \left(I + \Delta t \frac{dk_3}{dx(0)} \right)$$

$$\bullet \frac{dk_3}{dx(0)} = \dots$$

RK4 through back-propagation i.e. chain rule

Merging Physics and ANNs

Model Augmentation

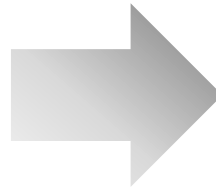
Solving PDEs with ANNs

Physics-Based Regularization

Model Augmentation

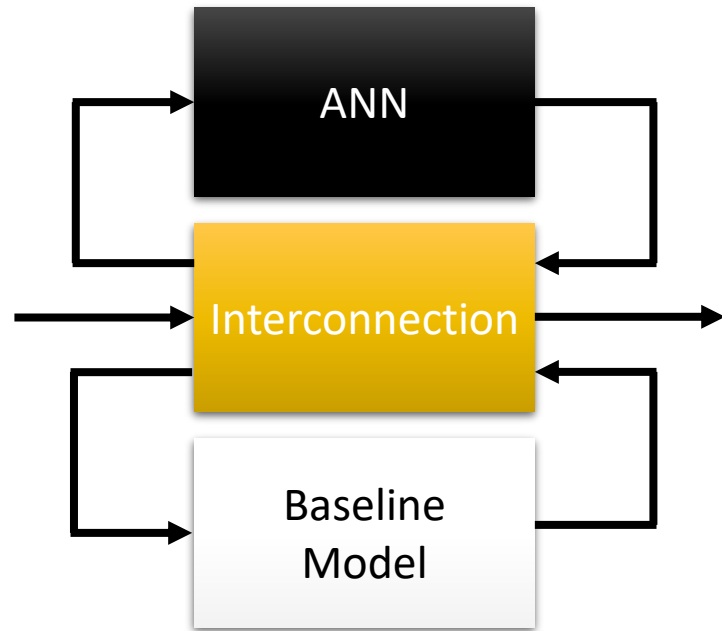


Baseline Model



Augmented Model

Model Augmentation



Baseline Model obtained from prior knowledge

Automatic Differentiation through the Baseline

Estimating Unknown dynamics with ANN

Example: JH Hoekstra, C Verhoek, R Tóth, M Schoukens. Learning-based model augmentation with LFRs, European Journal of Control, Volume 86A, 2025, 101304.

Solving PDEs using ANN

Observation about PDEs:

Expensive to solve

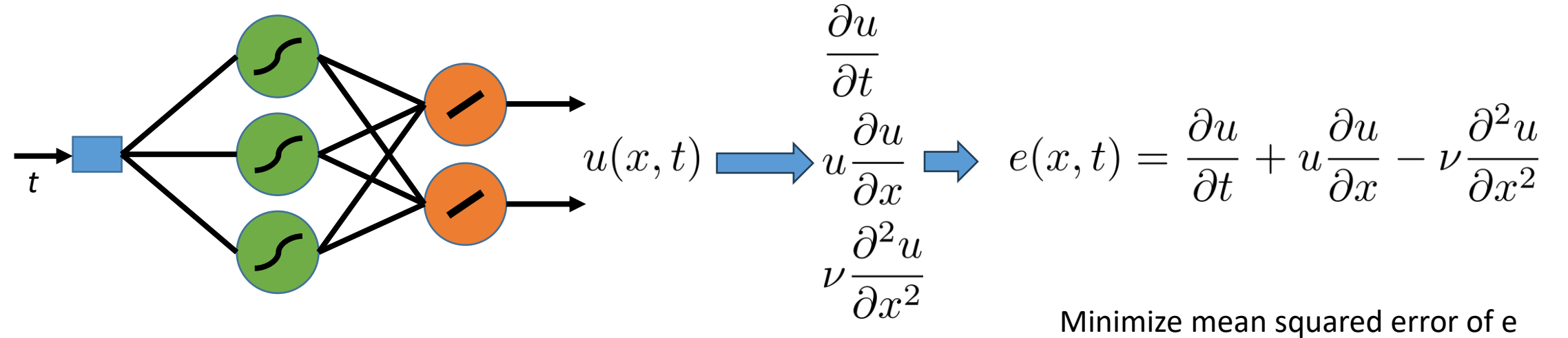
Easy to evaluate if a solution is correct

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

Burgers Equation

Idea: use ANN to solve PDE

Solving PDEs using ANN



$$V(\theta) = \frac{1}{TX} \sum_t \sum_x e(x, t)^2$$

Physics-Based Regularization

Introduce extra cost term

1. The first ‘classical’ term matches the model with the data

$$V_N^{[1]}(\theta) = \frac{1}{N_D} \sum_{k=0}^{N_D-1} (y_k^{[D]} - \hat{y}_k^{[D]})^2$$

2. A second ‘physics’ term matches the model with the physics

(e.g. through PINNS or mismatch with baseline model)

$$V_N^{[2]}(\theta) = \frac{1}{N_P} \sum_{k=0}^{N_P-1} (z_k^{[P]} - \hat{z}_k^{[P]})^2$$

$$V_N^{[tot]}(\theta) = V_N^{[1]}(\theta) + \gamma V_N^{[2]}(\theta)$$

Discussion

Embedding systems & control knowledge is advantageous, noise is important
a step towards explainable AI

Dynamic models have a wide range of use
control, system design, system validation, understanding

Model to be estimated can be
system model
feedforward controller / policy
feedback controller / policy

Artificial Neural Networks

Deep Learning & Deep Neural Networks

Training a Deep Neural Network

Artificial Neural Networks for Dynamical Systems